



Aras Innovator 33

Effectivity Services Programmer's Guide

Document #: D-008103

Last Modified: 8/22/2024

Copyright Information

Copyright © 2024 Aras Corporation. All Rights Reserved.

Aras Corporation
100 Brickstone Square
Suite 100
Andover, MA 01810
Phone: 978-691-8900

E-mail: support@aras.com

Website: <https://www.aras.com/>

Notice of Rights

Copyright © 2024 by Aras Corporation and/or its affiliates. All rights reserved.

This document is protected by U.S. and international copyright laws and conventions. No copyright may be obscured or removed from this document. This document may not be modified or altered, or reproduced or transmitted in any form, without the explicit permission of the copyright holder.

Aras Innovator, Aras, and the Aras Corp "A" logo are registered trademarks of Aras Corporation in the United States and other countries.

All other trademarks referenced herein are the property of their respective owners.

Notice of Liability

THIS DOCUMENT IS PROVIDED FOR INFORMATIONAL PURPOSES ONLY, AND THE CONTENTS HEREOF ARE SUBJECT TO CHANGE WITHOUT NOTICE. THE INFORMATION CONTAINED IN THIS DOCUMENT IS DISTRIBUTED ON AN "AS IS" BASIS, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE OR A WARRANTY OF NON-INFRINGEMENT. ARAS SHALL HAVE NO LIABILITY TO ANY PERSON OR ENTITY WITH RESPECT TO ANY LOSS OR DAMAGE CAUSED OR ALLEGED TO BE CAUSED DIRECTLY OR INDIRECTLY BY THE INFORMATION CONTAINED IN THIS DOCUMENT OR BY THE SOFTWARE OR HARDWARE PRODUCTS DESCRIBED HEREIN.

Table of Contents

Send Us Your Comments	6
1 Introduction	8
1.1 What is Effectivity Services?	8
2 Terminology	9
3 Getting Started	10
3.1 Effectivity Services Data Model Concept Overview	10
3.1.1 Effectivity Services Data Model Technical Overview	11
3.2 Effectivity Services Process Flow Overview	13
3.3 Process Flow Overview of Effectivity Services with Query Builder and Tree Grid View	15
4 Free-Form Boolean Expression Language	17
4.1 Expression Nodes	18
4.1.1 <code><expression>...</expression></code>	19
4.1.2 <code><eq>...</eq></code>	19
4.1.3 <code><ge>...</ge></code>	20
4.1.4 <code><le>...</le></code>	20
4.1.5 <code><variable id="..."></code>	20
4.1.6 <code><named-constant id="..."></code>	20
4.1.7 <code><constant>...</constant></code>	21
4.1.8 <code><and>...</and></code>	21
4.1.9 <code><or>...</or></code>	21
4.1.10 <code><not>...</not></code>	22
4.1.11 <code><implication>...</implication></code>	23
4.1.12 <code><exactly-one/></code>	24
4.1.13 <code><at-most-one /></code>	24
4.1.14 <code><at-least-one /></code>	25
4.2 Effectivity Criteria Expression Format	25
5 Scope Object Model	26
5.1 Scope Object Model Design	26
5.2 Customizing Scope Builder	27
5.2.1 Implementing the Scope Builder Method	27
5.2.2 Builder Method Usage	28
5.2.3 Caching	28
5.2.4 Sample Builder Method	29
6 Configuring Effectivity Services	31
6.1 Administrator Actions	31
6.1.1 Setting up Effectivity Scope	31

6.2	Administrator/User Actions.....	33
6.3	Configuring Effectivity Resolution	33
6.3.1	Creating a Query Definition to Filter by Effectivity.....	34
6.3.2	Creating a Tree Grid View to Display Effective Items	40
6.3.3	Adding “Set Effectivity Criteria” Button to the Tree Grid View Toolbar	48
6.3.4	Using Effectivity Criteria Filter Dialog for Resolving Effective Items	50
7	Dynamic Tree Grid Control API	60
7.1	Constructor.....	60
7.2	Public fields	60
7.3	Events	61
7.3.1	addRow.....	61
7.3.2	addRows.....	61
7.3.3	removeRow	62
7.3.4	removeAllRows.....	62
7.3.5	addColumn	62
7.3.6	removeColumn	62
7.4	Common objects description.....	63
7.4.1	Metadata objects	63
7.4.2	Column settings object.....	63
7.4.3	Column object.....	64
7.4.4	Row object.....	65
7.5	Public methods.....	65
7.5.1	loadData	65
7.5.2	getRowID	66
7.5.3	addRows.....	67
7.5.4	removeRow	68
7.5.5	removeAllRows.....	68
7.5.6	selectRow	68
7.5.7	deselectRow	68
7.5.8	addColumn	69
7.5.9	removeColumn	69
7.5.10	getChildRowIds	69
7.5.11	getRowUserData	70
7.5.12	setRowUserData	70
7.5.13	setCellEditability	71
7.5.14	getCellEditability	71
7.5.15	setCellValue	72
7.5.16	getCellValue	72
7.5.17	getColumnCount.....	72
7.5.18	getVisibleColumnCount.....	72
7.5.19	getColumnIndex	73
7.5.20	getColumnName.....	73
7.5.21	getColumnOrder	73
7.5.22	getAllColumnNames.....	73
7.5.23	getParentId	74

7.5.24	<i>containsRow</i>	74
7.5.25	<i>containsColumn</i>	74
7.5.26	<i>expandAll</i>	75
7.5.27	<i>collapseAll</i>	75
7.5.28	<i>isRowExpanded</i>	75
7.5.29	<i>isRowVisible</i>	75
7.5.30	<i>getVisibleRowCount</i>	75
7.5.31	<i>getAllRowCount</i>	76
7.5.32	<i>getVisibleRowIds</i>	76
7.5.33	<i>getAllRowIds</i>	76
7.5.34	<i>getRowId</i>	76
7.5.35	<i>getRowIndex</i>	76
7.5.36	<i>moveColumn</i>	77
7.5.37	<i>setColumnSettings</i>	77
7.5.38	<i>getColumnSettings</i>	77
7.5.39	<i>setColumnMetadata</i>	78
7.5.40	<i>getColumnMetadata</i>	78
7.5.41	<i>setColumnVisibility</i>	78
7.5.42	<i>getColumnVisibility</i>	78
7.5.43	<i>setColumnLabel</i>	79
7.5.44	<i>getColumnLabel</i>	79
7.5.45	<i>setColumnWidth</i>	79
7.5.46	<i>getColumnWidth</i>	80
7.5.47	<i>setCellMetadata</i>	80
7.5.48	<i>getCellMetadataOnly</i>	80
8	Appendix	82
8.1	How-to Example - Setting a default Effectivity Scope in “Effectivity Criteria Filter” dialog.....	82
8.2	How-to Example - Executing Query Definitions with Effectivity Resolution	84
8.2.1	<i>Retrieve Query Definition AML</i>	85
8.2.2	<i>Specifying Query Parameter Values for the Query Definition</i>	85
8.2.3	<i>Query Definition Execution</i>	87

Send Us Your Comments

Aras Corporation welcomes your comments and suggestions on the quality and usefulness of this document. Your input is an important part of the information used for future revisions.

- Did you find any errors?
- Is the information clearly presented?
- Do you need more information? If so, where and what level of detail?
- Are the examples correct? Do you need more examples?
- What features did you like most?

If you find any errors or have any other suggestions for improvement, indicate the document title, and the chapter, section, and page number (if available).

You can send comments to us in the following ways:

Email:

TechDocs@aras.com

Subject: Aras Product Documentation

Or,

Postal service:

Aras Corporation
100 Brickstone Square
Suite 100
Andover, MA 01810
Attention: Aras Technical Documentation

If you would like a reply, provide your name, email address, address, and telephone number.

If you have usage issues with the software, visit <https://www.aras.com/support/>

Document Conventions

The following table highlights the document conventions used in the document:

Convention	Description
Bold	Emphasizes the names of menu items, dialog boxes, dialog box elements, and commands. Example: Click OK.
Code	Code examples appear in <code>courier</code> font. It may represent text you type or data you read.
Yellow highlight	Code highlighted in yellow draws attention to the code that is being indicated in the content.
Yellow highlight with red text	Red text highlighted in yellow indicates the code parameter that needs to be changed or replaced.
<i>Italics</i>	Reference to other documents.
Note:	Notes contain additional useful information.
Warning	Warnings contain important information. Pay special attention to information highlighted this way.
Successive menu choices	Successive menu choices may appear with a greater than sign (-->) between the items that you will select consecutively. Example: Navigate to File --> Save --> OK .

1 Introduction

“Effectivity” refers to the ability to conditionally include items in a structure. This condition can consist of multiple variables influencing the decision to include or exclude an item in a specific configuration of the structure.

Let's use building a product structure as an example. This structure uses Part-1 for product model “X” and Part-2 for product model “Y.” In this case, product model is the variable used in defining the part's effectivity in this product structure.

There are many different types of effectivity variables depending on the types of products, for example:

- The Aerospace and Shipbuilding industries create low-volume, highly complex products. Differences in product configurations are tracked by variables, such as Model, Unit Number, or Hull Number.
- The Automotive and High-Tech industries create high-volume products. Configuration changes are influenced by when materials are available to use. Therefore, Date-based effectivity is commonly used.
- The Food and Beverage and Pharmaceutical industries create formula-based products. Changes to the product configuration are tracked using Lot, Batch, or Plant effectivity.

1.1 What is Effectivity Services?

Effectivity Services is built into the Aras Platform. It provides core functionality to develop applications to manage effectivity on structures and resolve structures for given effectivity criteria.

Together, Effectivity core services and the custom application layer enable you to:

- Define effectivity variables (such as date, model, unit, lot, batch, and plant)
- Set effectivity conditions on relationships
- Recursively resolve structures by effectivity to generate configured structures

2 Terminology

Table 1: Terms

Term	Definition
Effectivity	Identification of valid uses of an item in a structure if it is conditional.
Effectivity Variable	Variables that influence effectivity decisions, such as date, model, unit, batch, lot, plant.
Effectivity Scope	Built-in ItemType that represents a list of relevant Effectivity Variables.
Effectivity Expression	Expression representing the effectivity condition in the Boolean Expression Language. For example: Model = "Model X" and (Unit >= 10 and Unit <=20)
Effectivity Criteria	Criteria used for resolving a structure by effectivity. Effectivity conditions set on relationships are evaluated against the input criteria to determine the inclusion or exclusion of conditional items in the resolved structure. For example: Model = "Model X", Unit =15
Scope Object	Instance of the class "Aras.Server.Core.Configurator.Scope", and is a container of Variable instances.

3 Getting Started

Effectivity Services is used for resolving structures with effectivity conditions on Relationship Items for specified criteria.

The Aras Platform provides the following Effectivity Services features:

- Effectivity Services Data Model built with new ItemTypes, RelationshipTypes, Methods and more to enable customization and/or extension to support a custom flow.
- Effectivity Resolution Engine to determine which items are effective in a structure for a specific configuration.

3.1 Effectivity Services Data Model Concept Overview

The Effectivity Services data model concept contains the following main structural blocks:

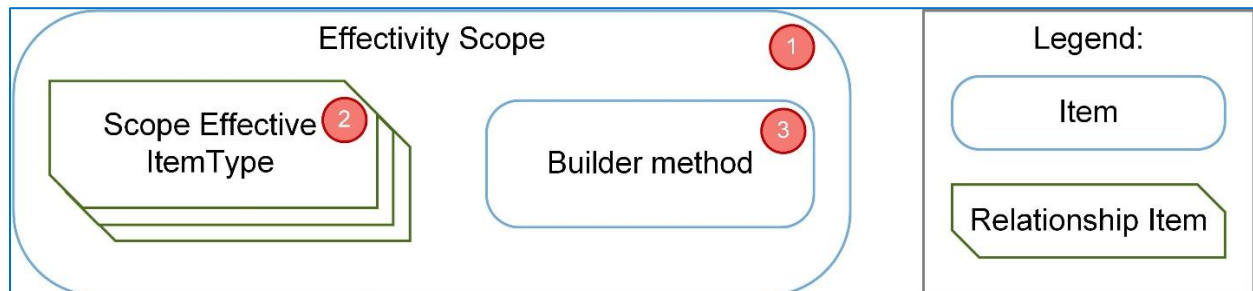


Figure 1.

1. **Effectivity Scope** is an item of the ItemType “effs_scope” used for defining the context for effectivity resolution. An Effectivity Scope item requires a reference to the “Builder method” and may have “Scope Effective ItemType” relations to the RelationshipTypes where effectivity is managed.
2. **Effectivity Scope ItemType** is an item of the RelationshipType “effs_scope_itemtype” which creates a relationship between an Effectivity Scope item and an ItemType (“RelationshipType” ItemType). When this relationship is established, the Effectivity Services Core automatically creates a new null (No Related) RelationshipType for the specified ItemType and names it according to the template “effs_%ITEMTYPE%_expression”, where %ITEMTYPE% is the ItemType name.

The RelationshipType “effs_%ITEMTYPE%_expression” is added as a poly source to the polymorphic ItemType “effs_expression” to subscribe to the Effectivity Resolution Engine. This RelationshipType retains Effectivity Expressions.

3. **Builder Method** is an item of the ItemType “Method”. A reference to it is stored in the “builder_method” property associated with the Effectivity Scope item. The Effectivity Resolution Engine calls this method to construct a Scope object, which serves as the base for effectivity resolution. For more information about the builder method, refer to section [5.2.1 Implementing the Scope Builder Method](#).

Note: Effectivity Scope and Scope Object are different. The Effectivity Scope is an item of the ItemType “effs_scope”. The Scope Object is an instance of the class “Aras.Server.Core.Configurator.Scope,” which is the result of Builder method execution.

3.1.1 Effectivity Services Data Model Technical Overview

This section contains a technical description of the Effectivity Services Data Model.

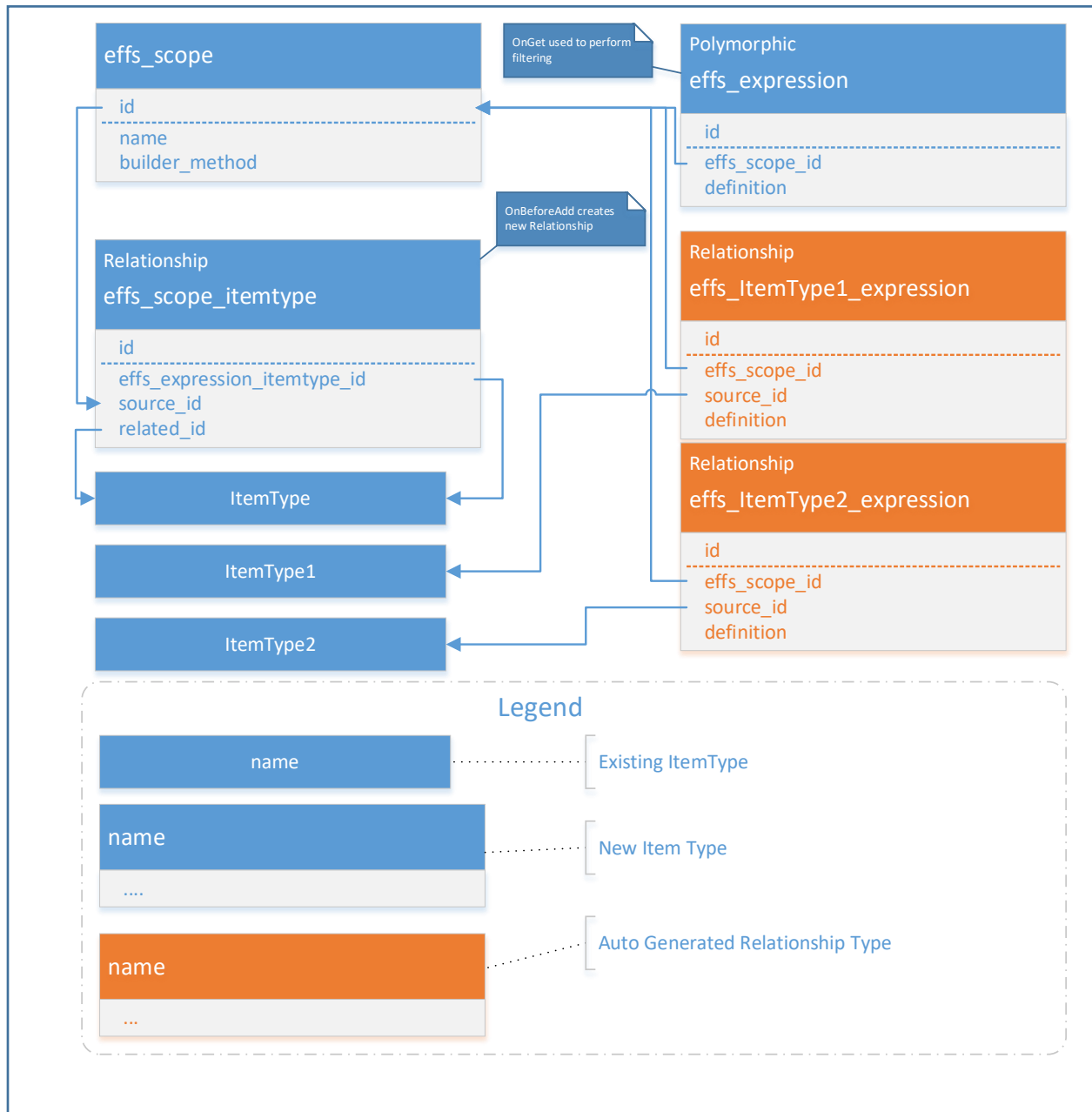


Figure 2.

Table 2: Effectivity Services Data Model

Item Type	Property	Description
ItemType		Standard ItemType ItemType, which is a holder of all ItemTypes in Aras Innovator
ItemType1, ItemType2		Relationship ItemTypes that are to be managed by effectivity.
effs_scope, effs_scope_itemtype, effs_expression		Effectivity Data Model
effs_scope		Container for scope object definition
	name	Name of Scope Item
	builder_method	Link to custom builder method that will construct Scope object for current Item
effs_scope_itemtype		Relationship between Scope (source_id) and target ItemType(related_id) onBeforeAdd event auto-generates no related relationship with name effs_#ItemType#_expression. Just one Item can be created for pair scope and item type.
	effs_expression_itemtype_id	ItemType id, effs_#ItemType#_expression
	source_id	effs_scope id
	related_id	ItemType id, target relationship ItemType
effs_expression		Polymorphic ItemType with onGet event for Effective Items filtering
	effs_scope_id	effs_scope id
	definition	Container for expression

Item Type	Property	Description
effs_#ItemType#_expression		Poly source for effs_expression. No related relationship to target ItemType
	effs_scope_id	effs_scope id
	source_id	item id for target Item Type
	definition	Container for expression

3.2 Effectivity Services Process Flow Overview

The following diagram shows the process flow for Effectivity Services.

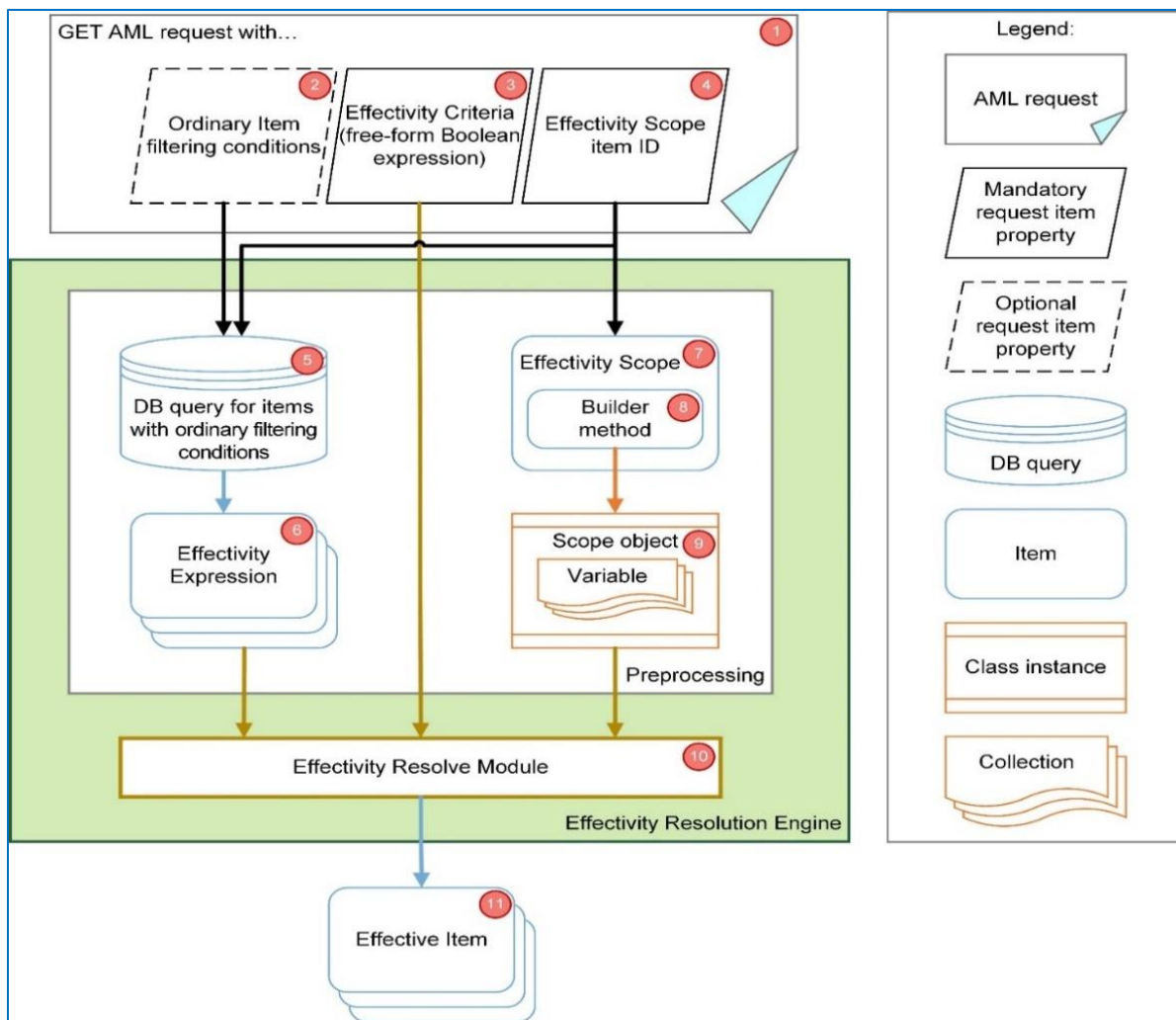


Figure 3.

1. **An AML request with the action “get”**, where there are mandatory and optional request item properties (nodes) to apply effectivity resolution on a requested structure.

Note: The Effectivity Resolution Engine intercepts only “GET” AML requests.

The current implementation of the Effectivity Resolution Engine can intercept “GET” AML requests to ItemTypes which are poly sources of the polymorphic ItemType “effs_expression”. Therefore, it can process items of poly source ItemTypes of the polymorphic ItemType “effs_expression” to determine effective items.

2. **Ordinary item filtering conditions** in an AML request are used as criteria to limit the items requested from a database (DB) in the preprocessing stage of effectivity resolution processing. The GetItem internal module uses them to request items from the DB to fulfill specified filtering conditions. For example, these criteria may be item id, a list of item ids (known as the “idlist” attribute on a request Item node), or item property tags in an AML request. Ordinary item filtering conditions are optional and provided as needed.
3. **Effectivity Criteria** is a condition (e.g., Model = Model-X and Production Date = 2021/06/28) written in the Free-Form Boolean expression. For more information, refer to section 4 Free-Form Boolean Expression Language. The criteria are used to evaluate each effectivity expression in a requested structure to determine effective items. An item is effective within a structure if the evaluation of its corresponding Effectivity Expression(s) against Effectivity Criteria produces the logical truth.

The representation of the “Effectivity Criteria” mandatory field in an AML request is the node “definition” matching the XPath “Item/definition | Item/and/definition” and containing a free-form Boolean expression in the XML-encoded form.

Note: The node “definition” is not passed to the internal GetItem module. Therefore, it will not affect the items requested from a database.

4. **Effectivity Scope item ID** is the unique identifier (ID) of an Effectivity Scope Item used as a context for effectivity resolution.

The representation of the “Effectivity Scope item ID” mandatory field in an AML request is the node “effs_scope_id” matching the XPath “Item/effs_scope_id | Item/and/effs_scope_id” and containing 32-hex GUID.

Note: The node “effs_scope_id” is passed to the internal GetItem module as part of querying the database.

5. **DB query for items** with ordinary filtering conditions is an action on the preprocessing stage of effectivity resolution. The internal GetItem module requests items from a DB with optional item filtering conditions and the mandatory property “effs_scope_id”. A response has items with effectivity expressions for evaluation with Effectivity Criteria.
6. **Effectivity Expression items** are the result of querying a DB for items, which fulfill specified optional item filtering conditions and the mandatory property “effs_scope_id”.
7. **Effectivity Scope** is an item of the ItemType “effs_scope” requested from a DB using the incoming mandatory request property “effs_scope_id”. This is to get a builder method referenced in the property “builder_method” of an Effectivity Scope item.
8. **Builder Method** is an item of the ItemType “Method”. The preprocessing stage of Effectivity Resolution extracts the last generation of the method code using a reference in an Effectivity Scope item, compiles the code and calls it with the single argument “Effectivity Scope Id” to return a properly constructed Scope object.

9. **Scope object** is a properly constructed instance of the class "Aras.Server.Core.Configurator.Scope", which has a list of Effectivity Variables, which will take a part in Effectivity Resolution.
10. **Effectivity Resolve Module** takes Effectivity Expression items, Effectivity Criteria, and a Scope object to evaluate each Effectivity Expression item against Effectivity Criteria. If this evaluation produces the logical truth, the item goes to the resulting structure.
11. **Effective items** are the result of the effectivity resolution process flow. The final structure contains only the items determined to be valid for the specified effectivity criteria.

3.3 Process Flow Overview of Effectivity Services with Query Builder and Tree Grid View

Combining Effectivity Services with the Query Builder and Tree Grid View applications is one of the native ways to visualize the results of the effectivity resolution process in the user interface.

The intended use of the Query Builder application is to create Query Definitions which are reusable AML requests. For more information about using Query Builder, refer to the *Aras Innovator 33 – Query Builder Guide*.

The Tree Grid View application enables you to build a visual data structure. For more information about Tree Grid View, refer to the *Aras Innovator 33 – Tree Grid View Administrator Guide*.

Effectivity Services acts as an interlayer in the native Query Definition process flow. The Effectivity Resolution Engine performs additional processing on the items requested by Query Definition.

Figure 4 illustrates the process flow for Effectivity Services combined with Query Builder and Tree Grid View.

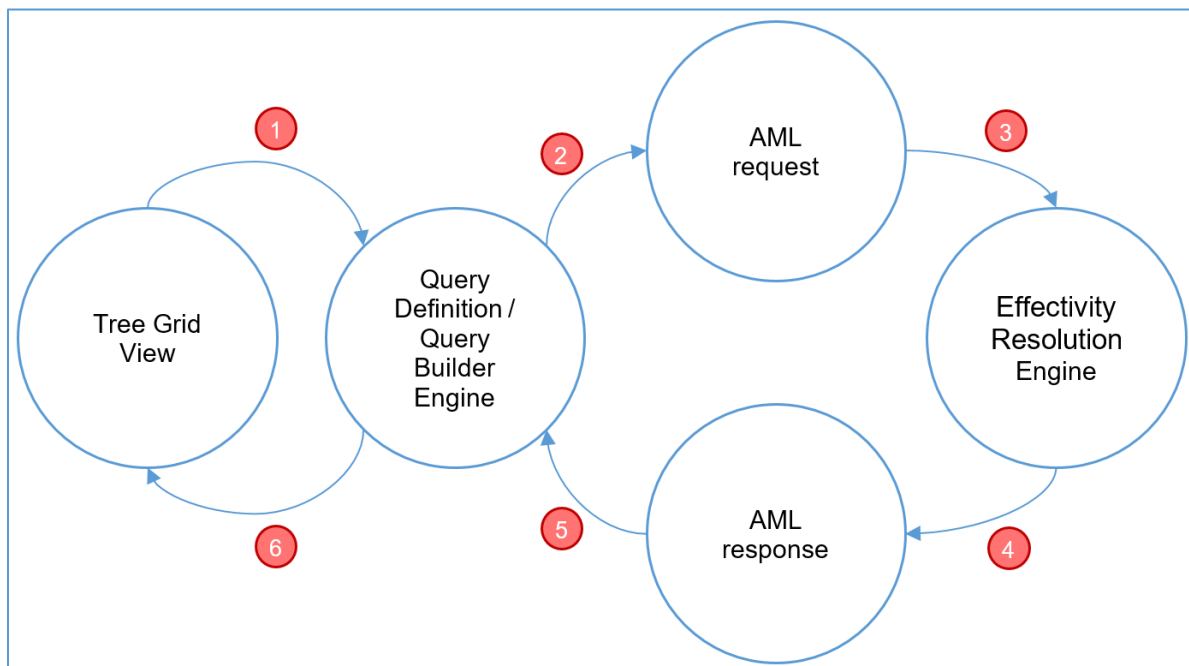


Figure 4.

1. The configured **Tree Grid View** uses its associated Query Definition to query data to display. For information on how to configure the Tree Grid View, refer to section [6.3.2](#) Creating a Tree Grid View to Display Effective Items.
2. The **Query Builder Engine** processes the received Query Definition to retrieve data from a database (DB) and return it. To complete this task, the Query Builder Engine prepares and sends the AML request internally. For information on how to configure the Query Definition, refer to section [6.3.1](#) Creating a Query Definition to Filter by Effectivity.
3. The **Effectivity Resolution Engine** receives the AML request from the Query Builder Engine, gets the necessary data from a DB by applying the request, and filters data according to the Effectivity Criteria.
4. The **Effectivity Resolution Engine** prepares the AML response containing the effective items and sends it back to the Query Builder Engine.
5. The **Query Builder Engine** receives and processes the requested data according to the requirements of the regular query execution flow.
6. The **Query Builder Engine** sends the items structure to the Tree Grid View to display.

4 Free-Form Boolean Expression Language

The Free-Form Boolean expression language enables you to easily define restrictions and relationships between Variables. It is based on the Aras Markup Language (AML). The following nodes are used in the language:

- expression
- eq
- ge
- le
- variable
- named-constant
- constant
- and
- or
- not
- implication
- condition
- consequence
- exactly-one
- at-most-one
- at-least-one

Some of these nodes contain required attributes, and some of them have a strict structure or required nodes.

When using an expression in AML, it should be represented as a value of a Container Node. This can be achieved in two ways: wrapped with CDATA or encoded.

CDATA Example:

```
<containerNode>
  <![CDATA[<expression>
    <and>
      <eq>
        <variable id="item_id_model" />
        <named-constant id="item_id_z5" />
      </eq>
      <eq>
        <variable id="item_id_unit" />
        <constant type="int">10</constant>
      </eq>
    </and>
  </expression>]]>
</containerNode>
```

Note: There can only be one CDATA node within a Container Node. The CDATA node can only contain one expression node.

Encoded Example:

```
<containerNode>
  &lt;expression&gt;
    &lt;and&gt;
```

```

    <eq>
      <variable id="item_id_model" />
      <named-constant id="item_id_z5" />
    </eq>
    <eq>
      <variable id="item_id_unit" />
      <constant type="int">10</constant>
    </eq>
  </and>
</expression>
</containerNode>

```

The following sections describe these nodes and include examples.

4.1 Expression Nodes

Table 4 lists dependencies between expression nodes.

Table 3: Dependencies between expression nodes

Tag	Can contain
expression	implication, and, or, not, eq, ge, le, exactly-one, at-most-one, at-least-one
eq	variable, named-constant, constant
ge	variable, constant
le	variable, constant
variable	id attribute
named-constant	id attribute
constant	type
and	implication, and, or, not, eq, ge, le, exactly-one, at-most-one, at-least-one
or	implication, and, or, not, eq, ge, le, exactly-one, at-most-one, at-least-one
not	implication, and, or, not, eq, ge, le, exactly-one, at-most-one, at-least-one
implication	condition, consequence
condition	implication, and, or, not, eq, ge, le, exactly-one, at-most-one, at-least-one
consequence	implication, and, or, not, eq, ge, le, exactly-one, at-most-one, at-least-one
exactly-one	eq

Tag	Can contain
at-least-one	eq
at-most-one	eq

4.1.1 <expression>...</expression>

The <expression> node is a root node that represents the expression. Use the <expression> tag to define an expression in Boolean language.

```
<expression>
  <and>
    <eq>
      <variable id="item_id_model" />
      <named-constant id="item_id_z5" />
    </eq>
    <le>
      <variable id="item_id_unit" />
      <constant type="int">10</constant>
    </le>
  </and>
</expression>
```

This example represents the Boolean expression for Model = Z5 AND Unit <= 10.

4.1.2 <eq>...</eq>

The <eq> node defines equivalence. Equivalence is defined between a Variable and a NamedConstant or Constant. <eq> has a strict content: it must include the node pair <variable> and <named-constant> or <constant>.

```
<eq>
  <variable id="item_id_model" />
  <named-constant id="item_id_z5" />
</eq>
```

This example represents the Boolean expression for Model = Z5.

```
<eq>
  <variable id="item_id_unit" />
  <constant type="int">10</constant>
</eq>
```

This example represents the Boolean expression for Unit = 10.

4.1.3 <ge>...</ge>

The <ge> node defines a greater than or equal to operator. This operator is defined between a Variable and a Constant. <ge> has a strict content: it must include the node pair <variable> and <constant>.

```
<ge>
  <variable id="item_id_unit" />
  <constant type="int">1</constant>
</ge>
```

This example represents the Boolean expression for Unit >= 10.

4.1.4 <le>...</le>

The <le> node defines a less than or equal to operator. This operator is defined between a Variable and a Constant. <le> has a strict content: it must include the node pair <variable> and <constant>.

```
<le>
  <variable id="item_id_unit" />
  <constant type="int">10</constant>
</le>
```

This example represents the Boolean expression for Unit <= 10.

4.1.5 <variable id="..." />

The <variable> node defines a variable element. This element is used to define the first part of an equivalence. The first part of an equivalence must be the variable. The "id" attribute is required; it defines the unique identifier for the instance.

```
<eq>
  <variable id="item_id_model" />
  <named-constant id="item_id_z5" />
</eq>
```

This example represents the Boolean expression for Model = Z5 where Model is the variable.

4.1.6 <named-constant id="..." />

The <named-constant> node defines the namedConstant element. This element is used to define the second part of an equivalence. The "id" attribute is required; it defines the unique identifier for the namedConstant instance. NamedConstant is used when a variable can have a value from a list of defined values.

```
<eq>
  <variable id="item_id_model" />
  <named-constant id="item_id_z5" />
</eq>
```

This example represents the Boolean expression for Model = Z5 where Z5 is the named constant.

4.1.7 <constant>...</constant>

The `<constant>` node defines a constant element. This element is used to define the second part of an operation. The “type” attribute is required; it defines the Constant value type. Supported types are as follows:

- Int
- DateTime
- String

```
<le>
  <variable id="item_id_unit" />
  <constant type="int">10</constant>
</le>
```

This example represents the Boolean expression for Unit <= 10 where 10 is the constant.

4.1.8 <and>...</and>

The `<and>` node defines the Boolean operation “and”. It can contain an unlimited number of allowed child tags.

Note: It is unnecessary to include the `<and></and>` node explicitly within the expression node because it is already implied.

```
<and>
  <eq>
    <variable id="item_id_model" />
    <named-constant id="item_id_z5" />
  </eq>
  <le>
    <variable id="item_id_unit" />
    <constant type="int">10</constant>
  </le>
</and>
```

This example represents the Boolean expression for Model = Z5 AND Unit <= 10.

4.1.9 <or>...</or>

The `<or>` node defines the Boolean operation “or”. It can contain an unlimited number of allowed child tags.

```
<or>
  <eq>
    <variable id="item_id_moel" />
    <named-constant id="item_id_z5" />
  </eq>
  <le>
    <variable id="item_id_unit" />
    <constant type="int">10</constant>
  </le>
```

</or>

This example represents the Boolean expression for Model = Z5 OR Unit <= 10.

The following is an example of an OR node using an inner AND node:

<or>

```
<eq>
  <variable id="item_id_model" />
  <named-constant id="item_id_z5" />
</eq>
<and>
  <ge>
    <variable id="item_id_unit" />
    <constant type="int">10</constant>
  </ge>
  <eq>
    <variable id="item_id_model" />
    <named-constant id="item_id_z6" />
  </eq>
</and>
```

</or>

This example represents the Boolean expression for Model = Z5 OR (Unit >= 10 AND Model = Z6).

4.1.10 <not>...</not>

The <not> node defines the Boolean operation “not”. Only one child node can be created within this node. The child node can contain an unlimited number of nested, allowed child nodes, as shown in the second example.

<not>

```
<eq>
  <variable id="item_id_model" />
  <named-constant id="item_id_z5" />
</eq>
```

</not>

This example represents the Boolean expression for NOT(Model = Z5).

The following example shows that the child node contained within the <not> node can have nested child nodes:

<not>

```
<and>
  <eq>
    <variable id="item_id_model" />
    <named-constant id="item_id_z5" />
  </eq>
  <eq>
    <variable id="item_id_unit" />
    <constant type="int">10</constant>
  </eq>
</and>
```

```
</and>
</not>
```

This example represents the Boolean expression for NOT(Model = Z5 AND Unit = 10). It is different than NOT(Model = Z5) OR NOT(Unit = 10).

4.1.11 <implication>...</implication>

The <implication> node defines a Boolean operation "implication", such as "if Model = Z6 then Unit >= 10". This node must contain the <condition> and <consequence> child nodes. The <condition> node must precede the <consequence> node. Neither of these nodes can be empty.

```
<implication>
  <condition>
    <eq>
      <variable id="item_id_model" />
      <named-constant id="item_id_z6" />
    </eq>
  </condition>
  <consequence>
    <ge>
      <variable id="item_id_unit" />
      <constant type="int">10</constant>
    </ge>
  </consequence>
</implication>
```

This example represents the Boolean expression for IF Model = Z6 THEN Unit >= 10.

You can have an unlimited number of child nodes in condition and consequence tags as shown in the example.

The following example shows that 'condition' and 'consequence' nodes can be used with inner 'and' and 'or' nodes:

```
<implication>
  <condition>
    <or>
      <eq>
        <variable id="item_id_model" />
        <named-constant id="item_id_z5" />
      </eq>
      <eq>
        <variable id="item_id_model" />
        <named-constant id="item_id_z6" />
      </eq>
    </or>
  </condition>
  <consequence>
    <and>
      <ge>
        <variable id="item_id_unit" />
        <constant type="int">10</constant>
```

```

        </ge>
      </le>
      <variable id="item_id_unit" />
      <constant type="int">20</constant>
    </le>
  </and>
</consequence>
</implication>

```

This example represents the Boolean expression for IF (Model = Z5 OR Model = Z6) THEN (Unit >= 10 AND Unit <= 20).

4.1.12 <exactly-one/>

The <exactly-one> node defines an operation. The operator describes a condition that means “one and only one of the listed equivalencies can be true”. <exactly-one> must contain a set of <eq> child nodes.

```

<exactly-one>
  <eq>
    <variable id="item_id_model" />
    <named-constant id="item_id_z5" />
  </eq>
  <eq>
    <variable id="item_id_model" />
    <named-constant id="item_id_z6" />
  </eq>
</exactly-one>

```

This example represents the Boolean expression for EXACTLY-ONE(Model = Z5 | Model = Z6).

Note: Each equivalence in EXACTLY-ONE must use the same variable.

4.1.13 <at-most-one />

The <at-most-one> node defines an operation. The operator describes a condition that means “either none or just one of the listed equivalencies can be true”. <at-most-one> must contain a set of <eq> child nodes.

Each equivalence in AT-MOST-ONE must use the same variable.

```

<at-most-one>
  <eq>
    <variable id="item_id_model" />
    <named-constant id="item_id_z5" />
  </eq>
  <eq>
    <variable id="item_id_model" />
    <named-constant id="item_id_z6" />
  </eq>
</at-most-one>

```

This example represents the Boolean expression for AT-MOST-ONE(Model = Z5 | Model = Z6).

Note: Each equivalence in AT-MOST-ONE must use same variable.

4.1.14 <at-least-one />

The <at-least-one> node defines an operation. The operator describes a condition that means “at least one of the listed equivalencies should be true”. <at-least-one> must contain a set of <eq> child nodes:

Each equivalence in AT-LEAST-ONE must use the same variable.

```
<at-least-one>
  <eq>
    <variable id="item_id_model" />
    <named-constant id="item_id_z5" />
  </eq>
  <eq>
    <variable id="item_id_model" />
    <named-constant id="item_id_z6" />
  </eq>
</at-least-one>
```

This example represents the Boolean expression for AT-LEAST-ONE(Model = Z5 | Model = Z6).

Note: Each equivalence in AT-LEAST-ONE must use the same variable.

4.2 Effectivity Criteria Expression Format

Effectivity Criteria supports only direct <eq> child nodes under <expression> node. It can have one or more <eq> nodes, but they cannot be grouped using any grouping operators. Otherwise, Effectivity Services core will throw an exception. Each equivalence node can contain only a Variable and a NamedConstant or Constant node.

```
<expression>
  <eq>
    <variable id="item_id_model" />
    <named-constant id="item_id_z5" />
  </eq>
  <eq>
    <variable id="item_id_unit" />
    <constant type="int">10</constant>
  </eq>
</expression>
```

5 Scope Object Model

5.1 Scope Object Model Design

The Scope object contains a list of Variables. A Variable can be one of three data types (integer, date, string) or can have Enum with a list of values that can be assigned to the variable.

Figure illustrates the structure of the Scope Object Model.

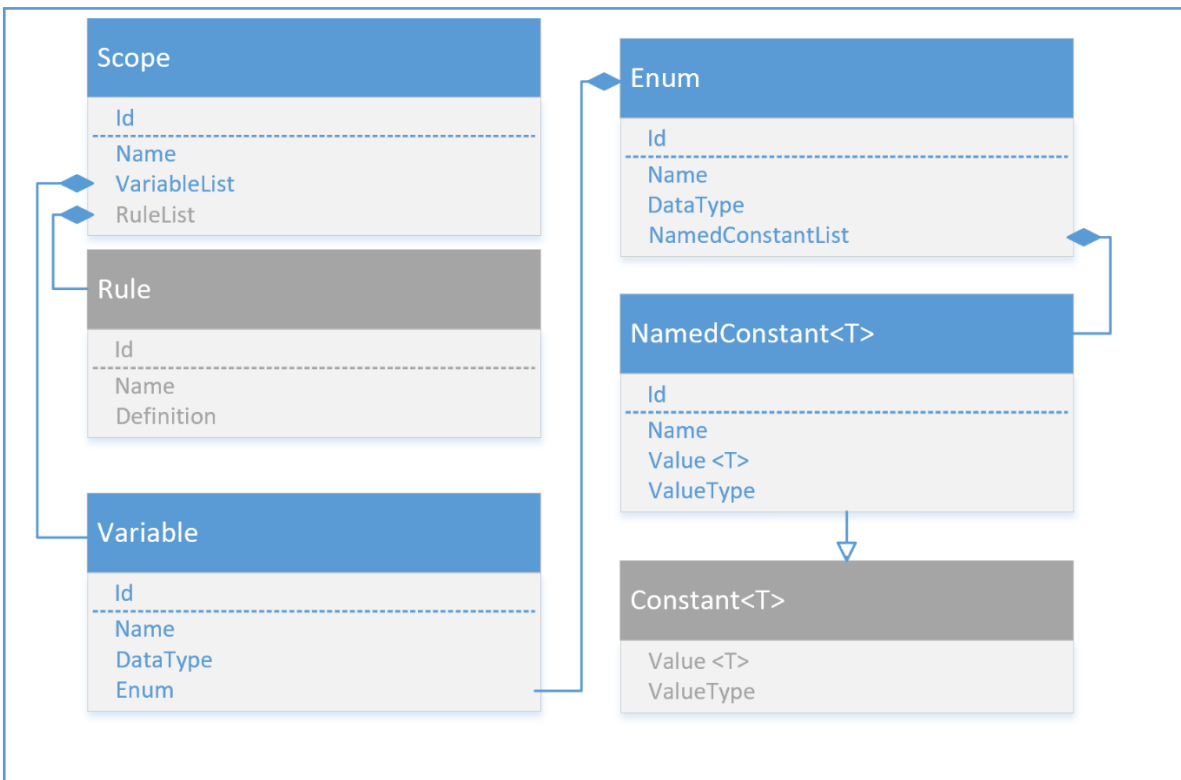


Figure 5.

Note: Effectivity Services does not support Rules. It means that RuleList should remain empty in the Scope object and Rule objects should not be created and added to the RuleList.

Variable classes store variable definitions. Variable datatypes are integer, date, string or null. The Variable type will be null if the Variable uses Enum with a list of Named constants. If Enum is null, the Variable is handled by type.

Enum is the container for the list of Named constants. A Named constant is intended to store the value definition. It contains the property name, value, and datatype.

5.2 Customizing Scope Builder

The Scope Builder method is a server method that is responsible for constructing a Scope Object. The purpose of the builder method is to construct a Scope Object using custom business data and business logic.

The Scope Builder uses a predefined method template. The template enables you to implement a Scope builder method properly.

5.2.1 Implementing the Scope Builder Method

The Scope Builder method uses the CSharp:Aras.Server.Core.Configurator template to define a class that inherits from the base abstract class. Here is the template for the Scope builder method:

```
namespace $(pkgname)
{
    public class $(clsname): ScopeBuilderBase
    {
        $(MethodCode)
    }
}
```

Base abstract class:

```
public abstract class ScopeBuilderBase
{
    public Item ScopeItem { get; set; }
    public IServerConnection IomConnection { get; private set; }
    public void Init(Item scopeItem, IServerConnection iomConnection) {}
    public abstract Scope BuildScope();
    public abstract string[] GetGuidsItemDependsOn();
    public abstract List<string> GetItemTypeNameItemDependsOn();
    public abstract ArrayList GetCustomKey();
}
```

The Scope Builder method is designed to provide a way for constructing a Scope object. The Scope Builder method also provides the built-in possibility to cache the Scope object.

Note: Implementing a Scope Builder method requires the override of all abstract methods - BuildScope, GetGuidsItemDependsOn, GetItemTypeNameItemDependsOn, GetCustomKey.

The BuildScope method is responsible for constructing the Scope object instance.

The GetGuidsItemDependsOn, GetItemTypeNameItemDependsOn, and GetCustomKey methods are used for Scope object caching and cache invalidation (see Caching section for more details).

Note: It is recommended to implement methods operating system agnostic for use with Linux and Windows. The primary OS incompatibility considerations for method implementation are path case-sensitivity, path separators, OS-specific line endings and OS-specific code. For more information about cross-platform development, please refer to section 2.3 *Cross-platform development* in *Aras Innovator 33 - Programmer's Guide*.

5.2.2 Builder Method Usage

The Effectivity Scope ItemType (effs_scope) is associated with the property builder_method. This property is a link to the Method Item. Effectivity Services automatically runs the linked builder method to get the Scope Object for the specified Effectivity Scope Item.

5.2.3 Caching

The Scope builder approach provides a built-in ability to cache a constructed Scope object. You can override the CustomKey function to store and retrieve the Scope object from the cache. This function returns the same ArrayList for each request for the same Scope object. To store different caches for different Scope objects, you can override the GetCustomKey function to return the different contents of the ArrayList:

```
public override ArrayList GetCustomKey()
{
    return new ArrayList {
        ScopeItem.getID(),
    };
}
```

The Scope object can be invalidated automatically. Invalidation is triggered if at least one of the Items that the Scope object depends on changes. To do this, the system maintains a list of Item IDs and list of ItemTypes associated with these Items. Any time an item associated with a maintained ItemType and ID is changed, the system invalidates the found item.

The following collections must be created to make caching and invalidation possible:

- **List of ItemType Names.** This list contains the name of each Item Type that is used to build the Scope.

```
public override List<string> GetItemNamesItemDependsOn()
{
    return new List<string> {
        "ItemType1",
        "ItemType2",
        "ItemType3",
        "ItemType4"
    };
}
```

Each Item Type in this list must be a poly source item for the **ScopeCacheDependency** Poly Item ItemType.

- **List of Item IDs.** This list contains the ID of each Item that is used to build the Scope.

```
public override string[] GetGuidsItemDependsOn()
{
    return new string[] {
        "item_id_Z4",
        "item_id_Z5",
        "item_id_Z6",
        "item_id_Z7"
    };
}
```

5.2.4 Sample Builder Method

The `effs_scope` ItemType has a property named `builder_method`. This Item property is the link to a custom server method. The Builder method is a server method that is responsible for constructing a Scope object from custom business data.

This sample shows how to implement a Builder method that creates a static Scope object.

```
//MethodTemplateName=CSharp:Aras.Server.Core.Configurator;
public override Scope BuildScope()
{
    Scope builtScope = new Scope { Id = "item_id_scope", Name = "Model/SN Scope" };
    Variable int_variable = new Variable(DataType.Int) { Id = "item_id_serialNumber", Name
= "Serial Number" };
    builtScope.VariableList.Add(int_variable);

    Aras.Server.Core.Configurator.Enum vEnum = new
Aras.Server.Core.Configurator.Enum(DataType.String) { Id = "item_id_modelList", Name =
"List of Models" };
    vEnum.AddNamedConstant("item_id_modelX", "Model X", "Model X");
    vEnum.AddNamedConstant("item_id_modelY", "Model Y", "Model Y");
    vEnum.AddNamedConstant("item_id_modelZ", "Model Z", "Model Z");
    builtScope.VariableList.Add(new Variable(DataType.String) { Id = "item_id_model", Name
= "Model", Enum = vEnum });

    return builtScope;
}

public override ArrayList GetCustomKey()
{
    return new ArrayList { "static_scope" };
}

public override string[] GetGuidsItemDependsOn()
{
    return new string[0];
}

public override List<string> GetItemTypeNamesItemDependsOn()
{
    return new List<string> { };
}
```

In this sample we have a static Scope object, so we can store it in the cache using a hardcoded ID as a key. The method result is cached with key `"static_scope"`.

The Scope has two Variables:

- [Serial Number] – data type of integer
- [Model] - data type of string, with list of values:
 - [Model X]
 - [Model Y]
 - [Model Z]

Note: IDs in this sample code are hardcoded to show the meaning of the builder method. An implementation of a builder method retrieves business data and builds the Scope object using actual ids.

GetCustomKey, GetGuidsItemDependsOn, and GetItemTypeNameItemDependsOn return either static or empty values for this sample static Scope Object. A Scope Object using real business data would return a feasible result.

Note: It is recommended to implement methods operating system agnostic for use with Linux and Windows. The primary OS incompatibility considerations for method implementation are path case-sensitivity, path separators, OS-specific line endings and OS-specific code. For more information about cross-platform development, please refer to section 2.3 *Cross-platform development* in *Aras Innovator 33 - Programmer's Guide*.

6 Configuring Effectivity Services

This section provides a brief description of what needs to be configured in Effectivity Services to use it with Query Builder and Tree Grid View. A more detailed description can be found in subsequent sections.

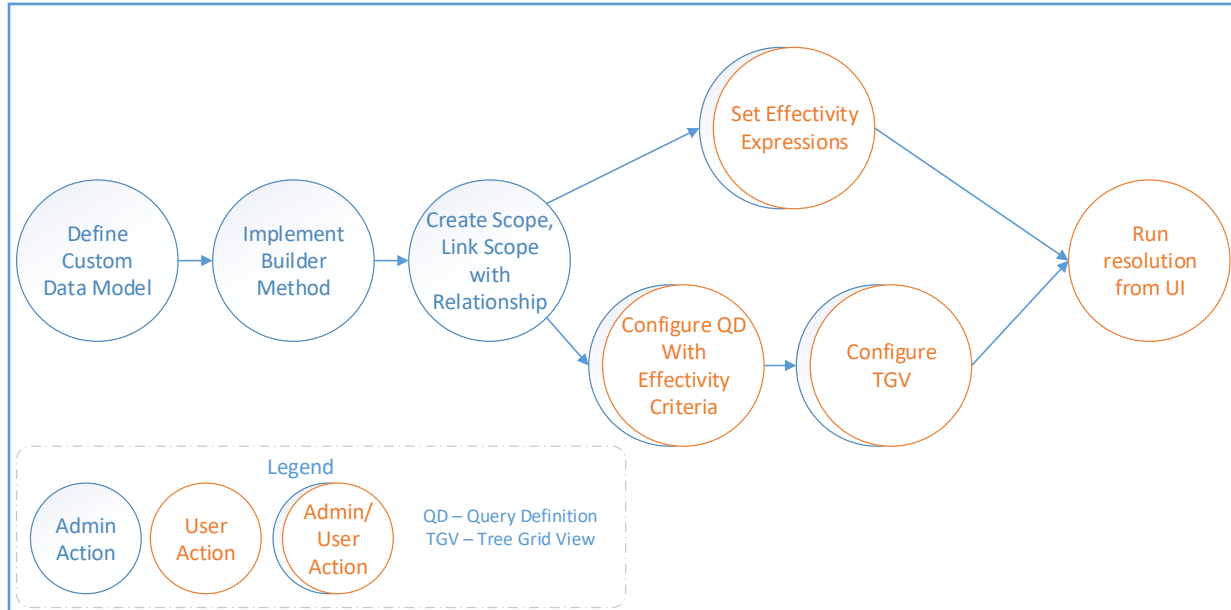


Figure 6.

6.1 Administrator Actions

Administrators are responsible for the following tasks:

- Define Custom Data Model – Define business item types to be used to build the Effectivity Scope. These can be item types for effectivity variables, relationships where effectivity information is managed.
- Implement builder method - Implement the server method that builds the Scope object from custom Data.
- Create Scope – Add effs_scope Item, specify Name and builder method. Link custom Data to Scope if needed.
- Link Scope with Relationship – Add the effs_scope_itemtype Item. Link Scope and Relationship ItemType that need to be managed by effectivity.

6.1.1 Setting up Effectivity Scope

This section describes how to configure Effectivity Scope to manage Effectivity on selected Relationship ItemTypes.

1. Create an Effectivity Scope item to define a context. Select **Administration → Effectivity Services → Effectivity Scope** in the TOC.

2. Select the Builder Method. Method `effs_scopeObjectBuilderMethod` is provided out of the box and is used to construct a Scope object based on related Effectivity Variables.
3. Select Effectivity Scope ItemTypes where effectivity will be managed.
4. Select Effectivity Variables for the Scope by creating or using existing Effectivity Variables.
5. Save the Effectivity Scope.

The screenshot displays the 'Effectivity Scope' configuration window for a '3D Printer'. The 'Name' field is set to '3D Printer' and the 'Builder Method' is 'effs_scopeObjectBuilderMethod'. The 'Effectivity Scope ItemType' tab is selected, showing a table with one item: 'Part BOM'. The table has columns for 'Name' and 'Description [...]'. Navigation buttons like 'Prev', 'Next', and 'Page: 1 of 1' are at the bottom.

Figure 7.

The new “Effectivity” (`effs_EffectivityScopeItemType_expression`) “No Related” relationship ItemType is created automatically. With this configuration, an “Effectivity” tab appears on the *EffectivityScopeItemType* form.

Note: You can have any number of different Relationships in the same Effectivity Scope.

If the automatically generated RelationshipType name (`“effs_%ITEMTYPE%_expression”`) is longer than 30 characters, you will not be able to create the relationship in the relationship grid. In this case, you can either manually enter the name on the **Effectivity Scope ItemType** form (by selecting Effectivity Scope ItemType, RMB→Open) or send an AML request with the explicitly specified property “name” to create the RelationshipType. The length of the property “name” value should not exceed 13 characters.

6. Log out of Aras Innovator and login again as new relationship ItemTypes are created automatically unless they already exist.

6.2 Administrator/User Actions

Both Administrators and Users can perform the following actions:

- Set Effectivity Expressions – Create the effs_#ItemType#_expression Item with an Effectivity Expression for a selected Relationship Item.
- Configure Query Definition to work with Effectivity Criteria – Create a Query Definition that selects your data using the specified criteria expression.
- Configure Tree Grid View to display returned data – Create a Tree Grid View configuration that shows the data returned from executing the Query Definition.

6.3 Configuring Effectivity Resolution

Using Query Builder with Effectivity Services, resolved item structures can be presented in a Tree Grid View. For additional information about Query Builder, refer to the *Aras Innovator 33 – Query Builder Guide*. For additional information about Tree Grid View, refer to the *Aras Innovator 33 – Tree Grid View Administrator Guide*.

Note: In this guide, the Part BOM ItemType is used as an example to explain Effectivity Services. The user interface (UI) does not reflect the actual UI in the Product Engineering application.

This section describes the following use cases using Effectivity Services combined with Query Builder and Tree Grid View:

- Resolving a Part BOM structure for the given effectivity criteria.
- Viewing all parts in a Part BOM structure that are a) effective for the criteria contained in the specified Effectivity Scope and b) have no Effectivity Expressions in the specified Effectivity Scope.

Before you can configure a Query Definition and a Tree Grid View, you need to:

- Define the business data that will be converted to Variables and Named Constants by the Builder Method.
- Implement the Builder Method.
- Create the Effectivity Scope.
- Relate a Relationship ItemType to the Effectivity Scope instance.

To follow the example in this section the Administrator has already defined the following structural elements:

- The Effectivity Scope item “3D Printer”. The Effectivity Scope item has the Relationship ItemType “Part BOM” as the Effectivity Scope ItemType.
- The Effectivity Variables associated with the Effectivity Scope item:
 - Model – is an Item Variable with effs_model ItemType. “Replicator” and “Replicator 2” items were created in the **Administration → Effectivity Services → Models** TOC category which means this Effectivity Variable has only two possible Named Constants (“Replicator” and “Replicator 2”), which can be assigned to it.

- Unit – is an Integer Variable. That means any appropriate integer value can be assigned to this Effectivity Variable.
- Production Date – is a Date Variable. That means any appropriate date value can be assigned to this Effectivity Variable.

6.3.1 Creating a Query Definition to Filter by Effectivity

A Query Definition is a fundamental element to retrieve data from the server.

In this section we use the **Part → Part BOM** structure to show how Effectivity Services can be used.

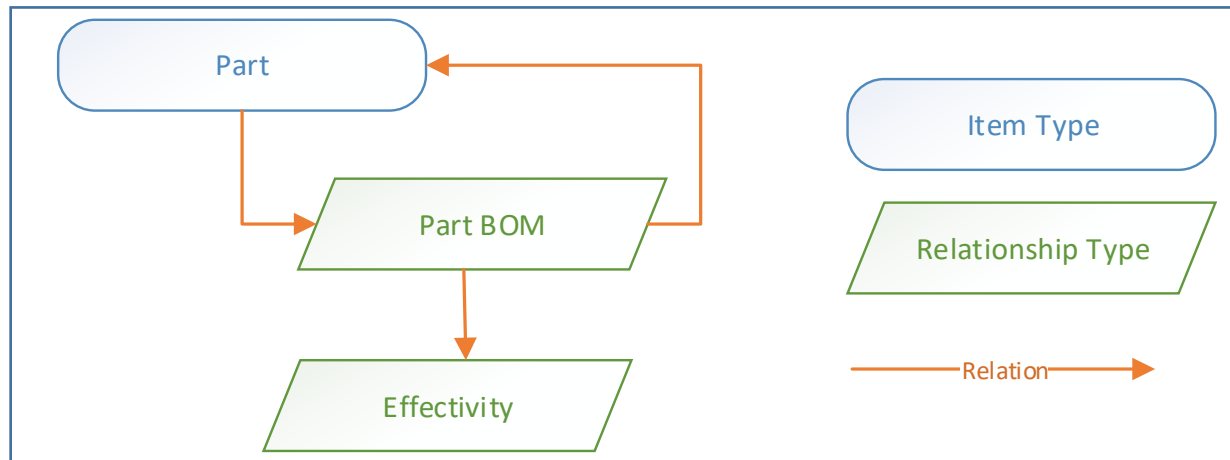


Figure 8.

6.3.1.1 Creating the Query Definition

1. Select **Administration → Configuration → Query Definitions** in the TOC.
2. Click **Create New Query Definition**.
3. Enter a value in the **Name** field.

- Click on the ellipses in the Context ItemType dialog to search for a **Context Item Type**. Once you select the ItemType and save the form, the selected ItemType appears as a link as shown here.

The screenshot shows a web application window titled "Part-Structure". The main area is labeled "Query Definition". It contains a form with the following fields:

- Name:** A text input field containing "Part-Structure".
- Context Item Type:** A dropdown menu showing "Part" with a link icon.
- Description:** A large text area for additional information.

The top toolbar includes buttons for "Edit", "Refresh", "Undo", "Redo", "Save", "Print", and a menu icon. The left sidebar shows a list of icons, including a camera icon.

Figure 9.

- Click the **Show Editor** button on the left sidebar to switch to Query-Editing Mode. Build a recursive Part structure using the Part BOM relationship.

The screenshot shows the same "Part-Structure" dialog, but now in "Query-Editing Mode". The top toolbar has changed to include "Save", "Done", and "Discard" buttons. The left sidebar now shows a "Show Editor" button. The main area displays a hierarchical structure of the Part BOM relationship:

- Part** (gear icon)
 - [Part.id = [Part BOM].source_id] Part BOM** (link icon)
 - [[Part BOM].related_id = Part.id] Part** (gear icon)

Figure 10.

6. Select the properties that you want the query to return. Let's select the following properties for Part:

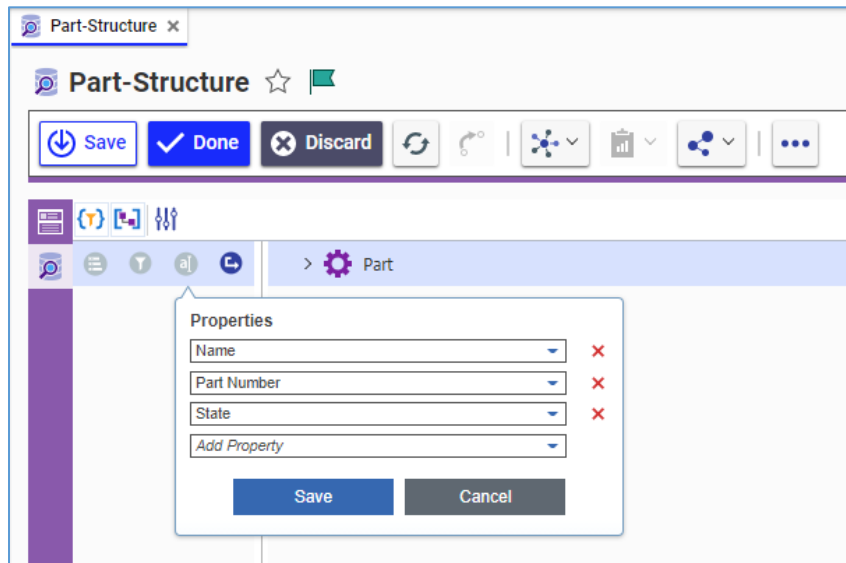


Figure 11.

6.3.1.2 Defining Effectivity Criteria

To define effectivity criteria, add the corresponding "Effectivity" relationship (effs_Part_BOM_expression), where Effectivity Conditions are saved.

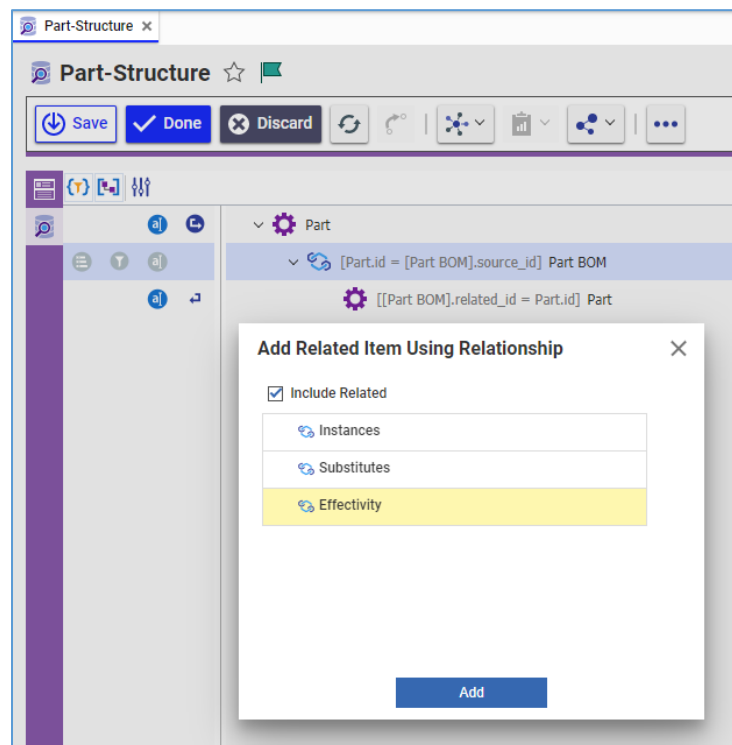


Figure 12.

1. Add the **Effectivity** relationship twice with different aliases as shown in [Figure 13](#).
2. Use the Relationship with the **effs_Part_BOM_expression** alias for structure resolution by effectivity.
3. Use the Relationship with the **effs_Part_BOM_expr_no_resolve** alias to check if the Part BOM item has any "Effectivity" relationship Items for the specified Scope.

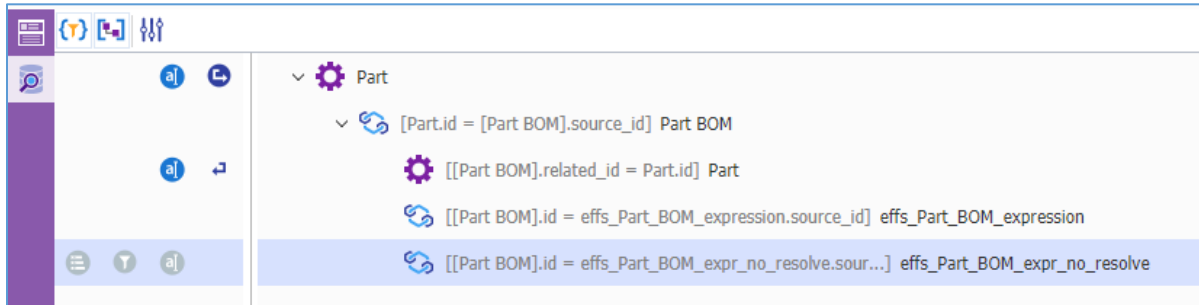


Figure 13.

These two aliases will be used in the where condition on the **Part BOM** query item.

4. Add a where condition to the "Part BOM" query item. The "(Count(effs_Part_BOM_expr_no_resolve) = 0) OR (Count(effs_Part_BOM_expression) > 0)" condition allows us to get the related part in case the Part BOM item does not have any effectivity relationships (Count(effs_Part_BOM_expr_no_resolve) = 0), or it has at least one effectivity expression resolved to true for the given effectivity resolution criteria/condition (Count(effs_Part_BOM_expression) > 0).

Note: It is recommended to use only the "Count" aggregate function for effectivity relationships to prevent performance degradation.



Figure 14.

5. Add a where condition "Scope = \$EffectivityScopeId" on the "effs_Part_BOM_expr_no_resolve" query item filtering by the "Scope" property.

The "Scope" value must match the ID of the scope which is provided via \$EffectivityScopeId Query Parameter created for this Query Definition. Query parameters allow us to execute the same query using different parameter values.

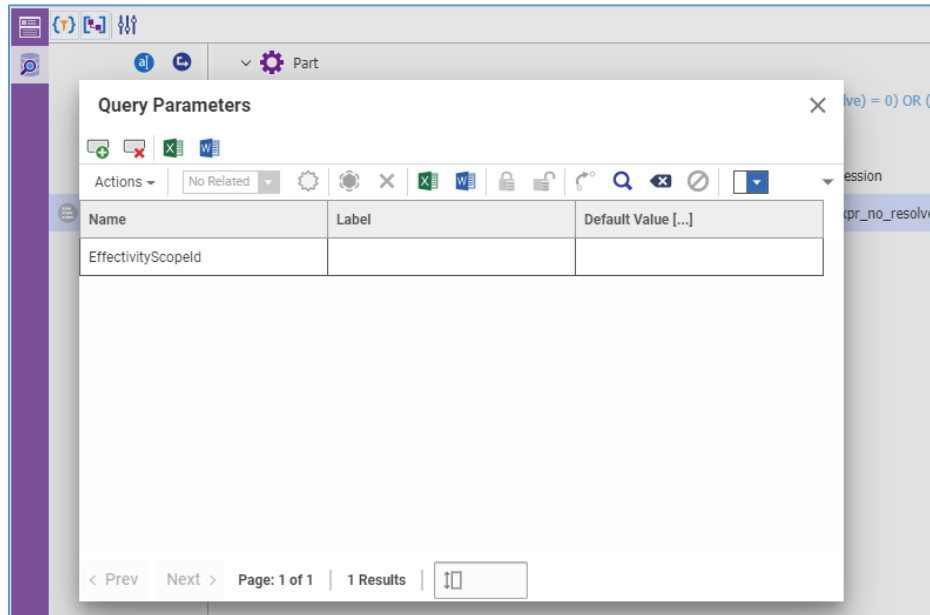


Figure 15.

Note: Default Value can be provided for the \$EffectivityScopeId Query Parameter. The Parameter name should be exactly "EffectivityScopeId" to be compatible with Effectivity Resolution via Effectivity Criteria Filter dialog.

This condition gives us Part BOMs that have no configured Effectivity Expression in the current Scope.

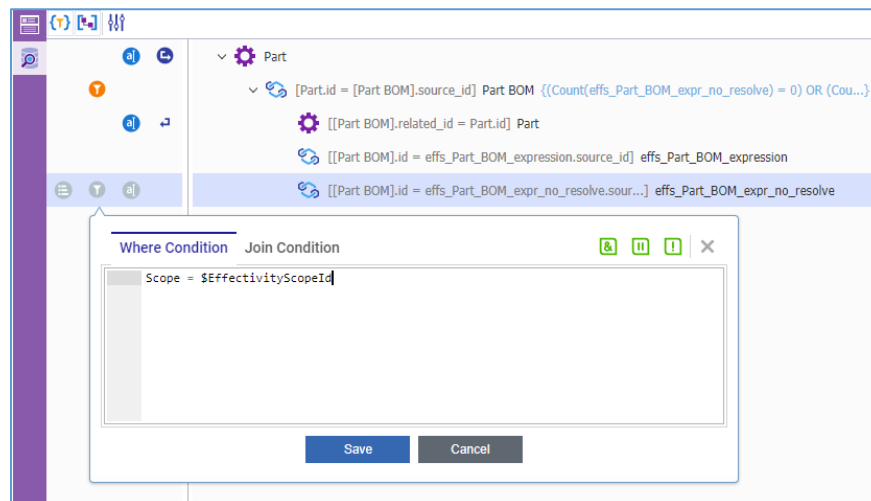


Figure 16.

6. Add a where condition "(Scope = \$EffectivityScopeld) AND (Definition = '<expression>\$EffectivityCriteriaDefinition</expression>')" on the "effs_Part_BOM_expression" query item filtering by the "Scope" and "definition" properties.

The "Scope" value must match the ID of the scope which is provided via \$EffectivityScopeld Query Parameter created for this Query Definition.

The Definition property value must contain an effectivity expression with Effectivity Criteria which is provided via \$EffectivityCriteriaDefinition Query Parameter created for this Query Definition.

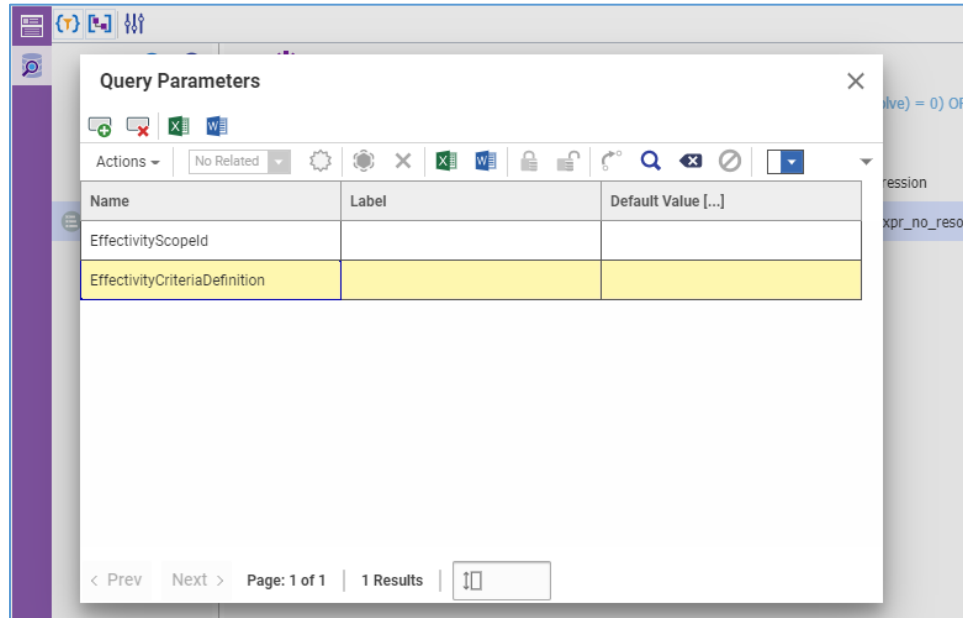


Figure 17.

Note: The Parameter name should be exactly "EffectivityCriteriaDefinition" in order to be compatible with Effectivity Resolution via the Effectivity Criteria Filter dialog.

This condition gives us Part BOMs that have a configured Effectivity Expression in the current Scope. The configured Effectivity Expression corresponds to the Effectivity Criteria from the “definition” property of the Query Item.

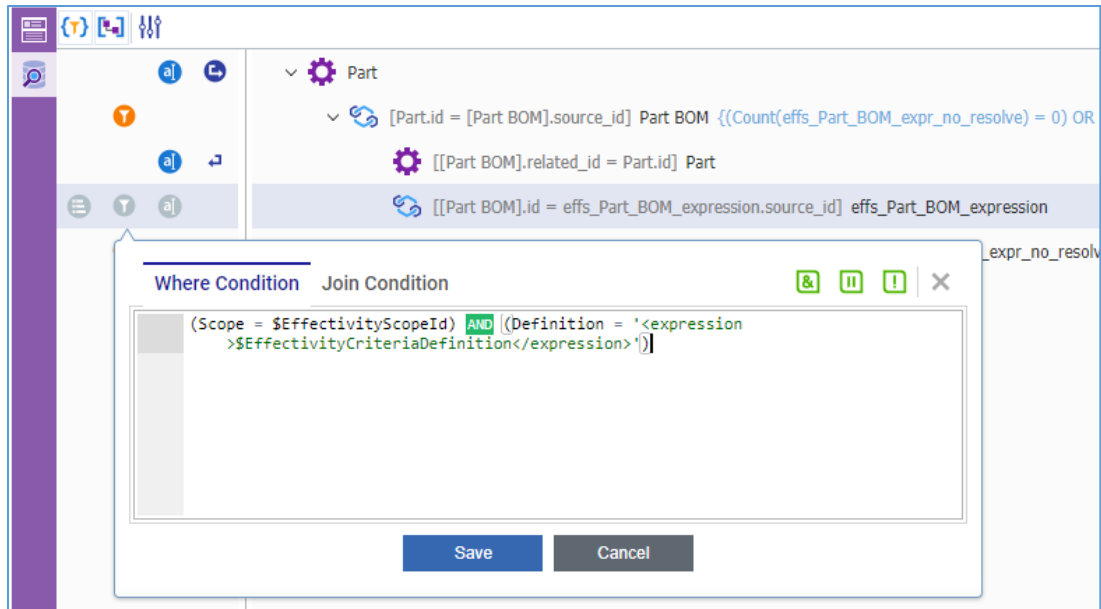


Figure 18.

7. Confirm that all conditions are set correctly. The final version of the effectivity query definition is as follows:

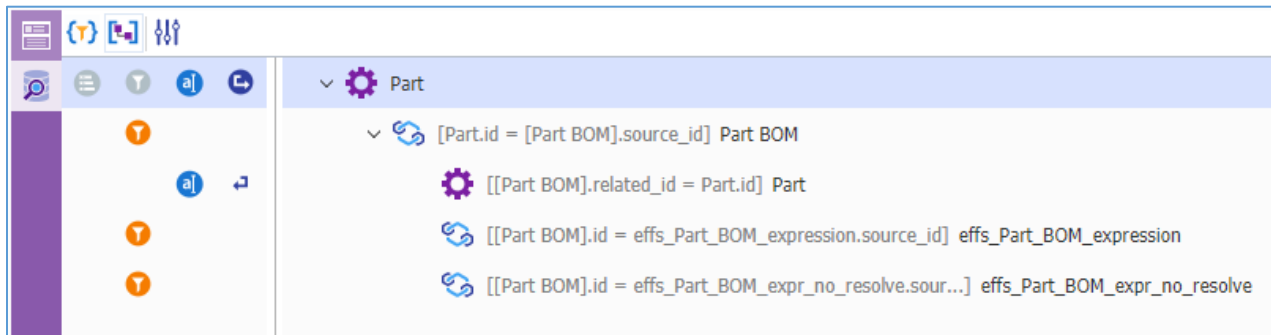


Figure 19.

6.3.2 Creating a Tree Grid View to Display Effective Items

Tree Grid View is standard functionality for Aras Innovator. It offers a visual layout of the data as a Relationship tab in an item view. In this example Tree Grid View uses the Query Definition created in the previous section to resolve the Part BOM structure and display the result in the grid.

6.3.2.1 Creating a Tree Grid View Item

1. Go to **TOC → Administration → Configuration → Tree Grid Views**.
2. Click **Create New Tree Grid View**.
3. Enter a value in the **Name** field.

- Specify the query definition in the **Query Definition** field.

Part Structure

Part Structure ☆

Edit Refresh Undo

Tree Grid View

Name: Part Structure Query Definition: Part-Structure

Context Item Type: Part

Description:

Max Visible Children On Expand: 100

Max Grow Levels: 3

Auto Grow On Refresh: ☒

Linked Toolbar/Context Menu:

Figure 20.

- Click the **Show Editor** button on the sidebar.

Part Structure

Part

Part BOM

Part

effs_Part_BOM_expression

effs_Part_BOM_expr_no_resolve

Figure 21.

6. Select the "Part", "Part BOM" and nested "Part" rows, right-click and select **Map Element** from the context menu to include these elements in the tree grid view.

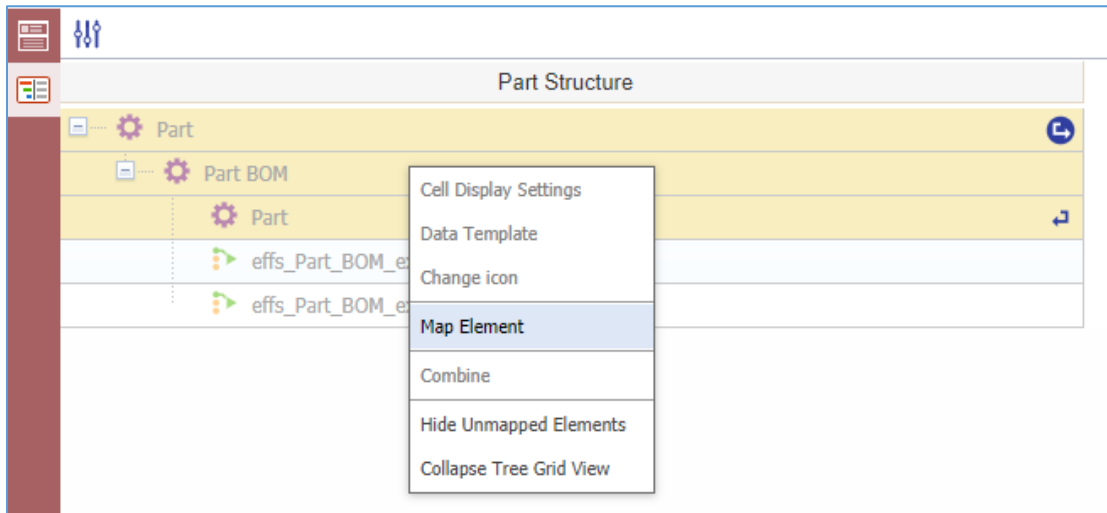


Figure 22.

7. Select Part BOM and nested Part elements, right click on them and choose **Combine** from the context menu to combine them into one element for better structure visualization in the Tree Grid View.

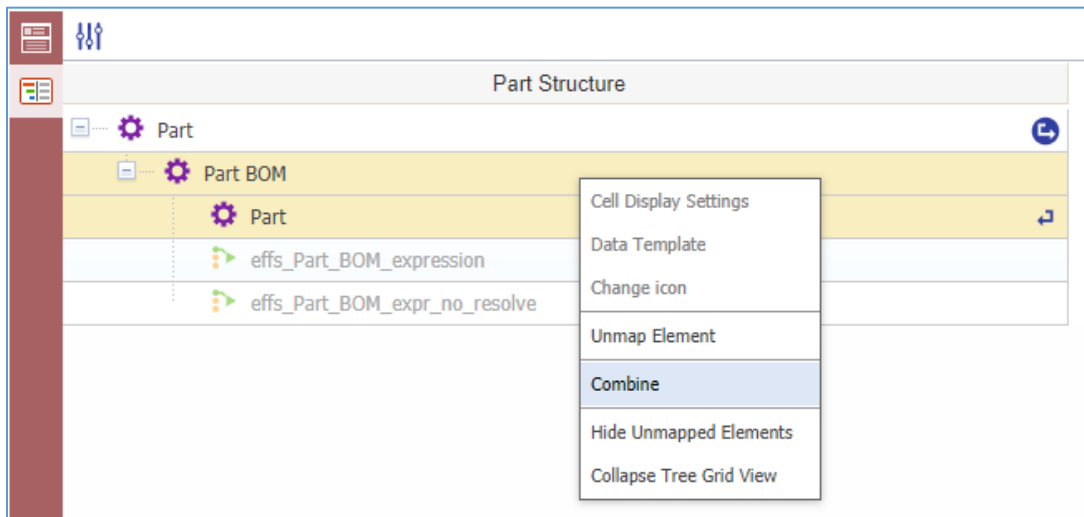


Figure 23.

8. Select the row with combined Part BOM and Part elements, right click on it and choose **Change icon** from the context menu to change the standard icon for the combined row.

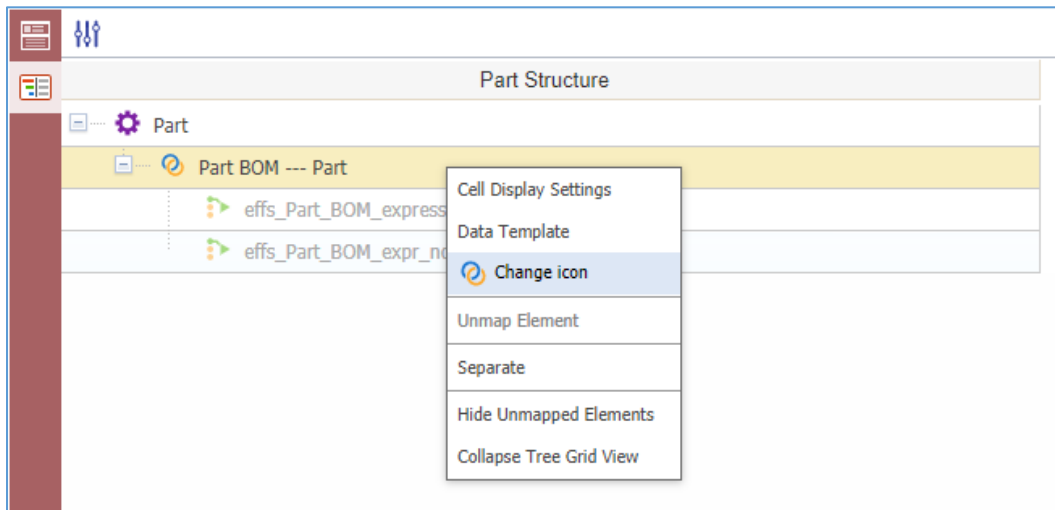


Figure 24.

9. Right click on the **Part Structure** column header and select **Add New Column**.

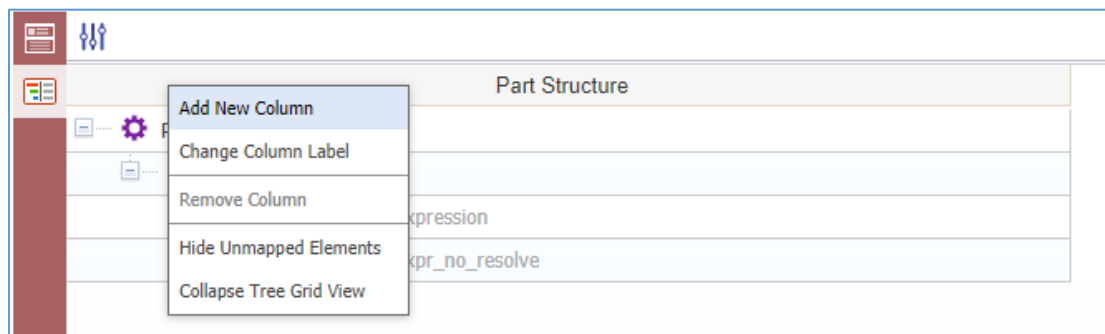


Figure 25.

10. Change the new column label to **State**.

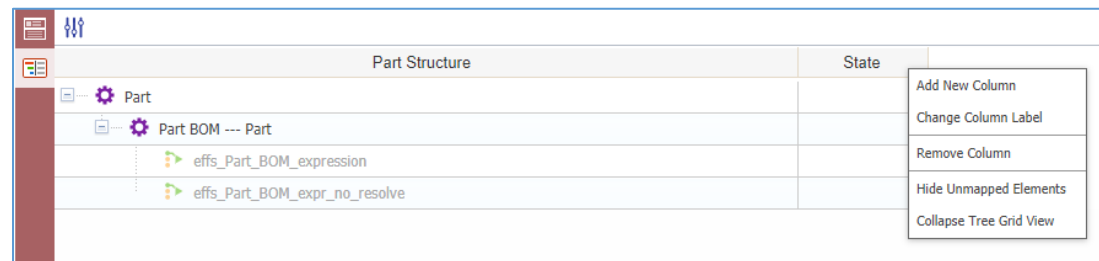


Figure 26.

11. Select the **State** cell in the Part row, right click on it and choose **Cell Display Settings** from the context menu.

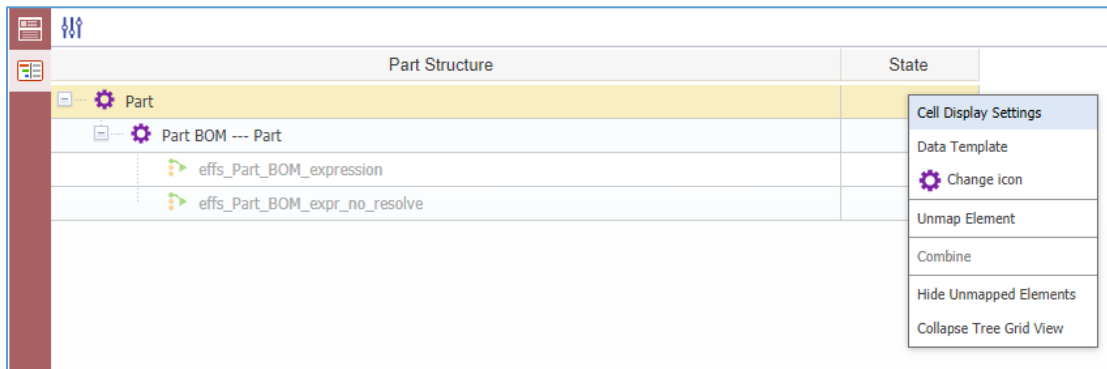


Figure 27.

12. Select the **{Part.state}** option in the Helper listbox to set it as the text template for this cell.

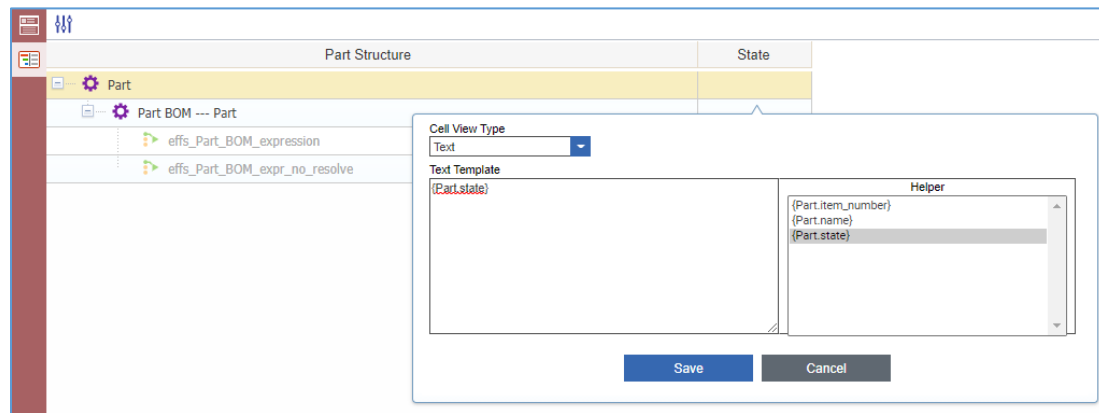


Figure 28.

13. Set **{Part.state}** as the text template for the "State" cell in the "Part BOM --- Part" row.

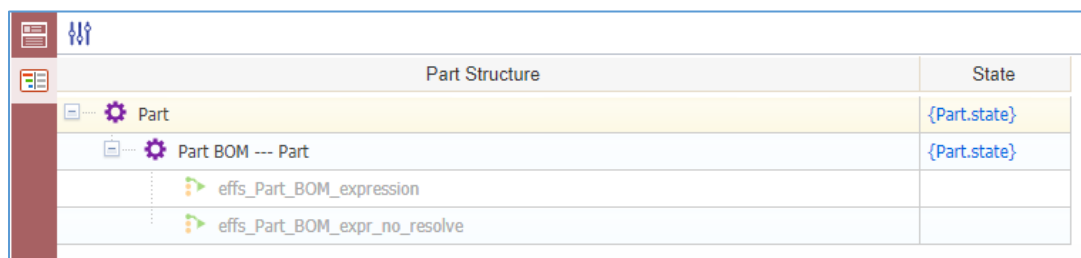


Figure 29.

14. Add the **Part Number** column and setup display settings as shown here:



Part Structure	Part Number	State
Part {Part.name}	{Part.item_number}	{Part.state}
Part BOM --- Part {Part.name}	{Part.item_number}	{Part.state}
effs_Part_BOM_expression		
effs_Part_BOM_expr_no_resolve		

Figure 30.

15. Click **Save** to save your changes.

6.3.2.2 Tree Grid View Usage

1. From the Tree Grid View, click the Actions menu and select **Set Tree Grid View Usage**.

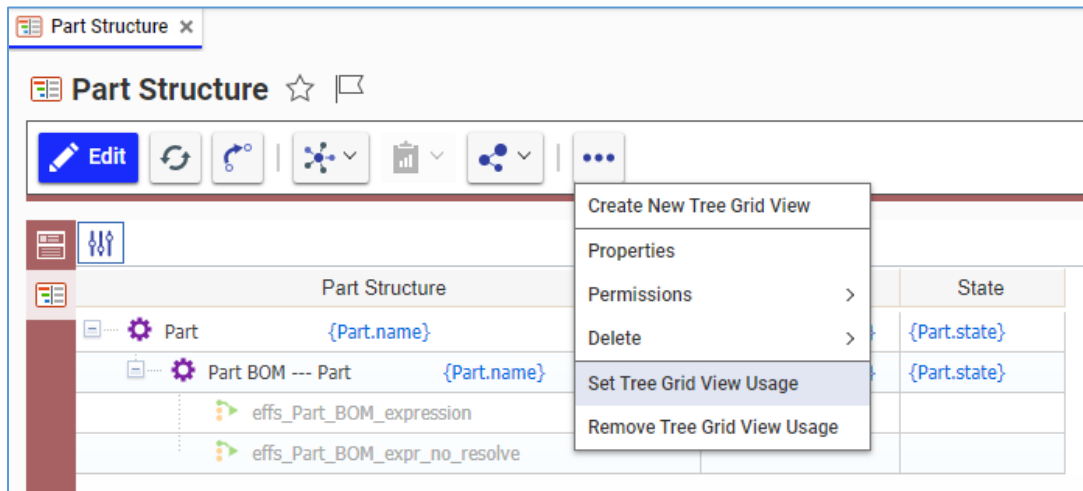


Figure 31.

2. Choose the **Relationship Tab** option as the target usage in the Set Tree Grid View Usage dialog box.

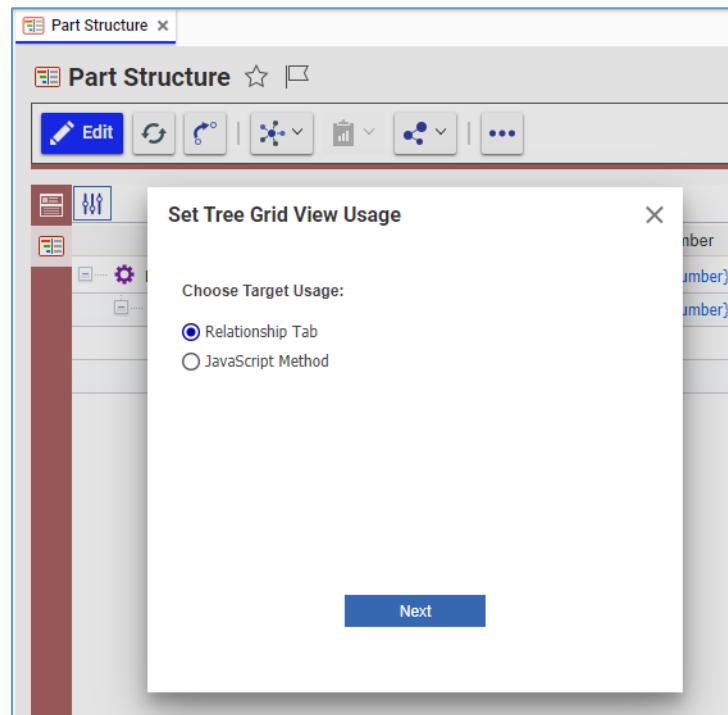


Figure 32.

3. Click **Next** to proceed to the next dialog box. Click the ellipses in the **Used On** field and select the ItemType to add to the new relationship. Click **Next**.

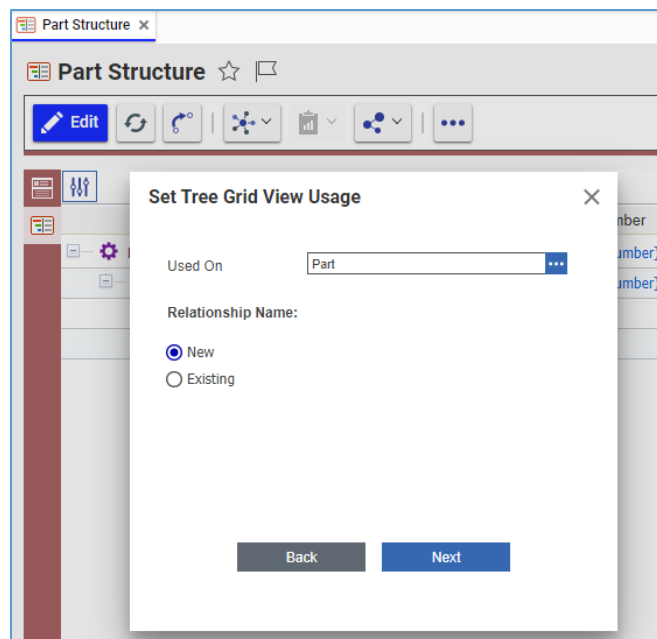


Figure 33.

4. Specify the relationship name and label.

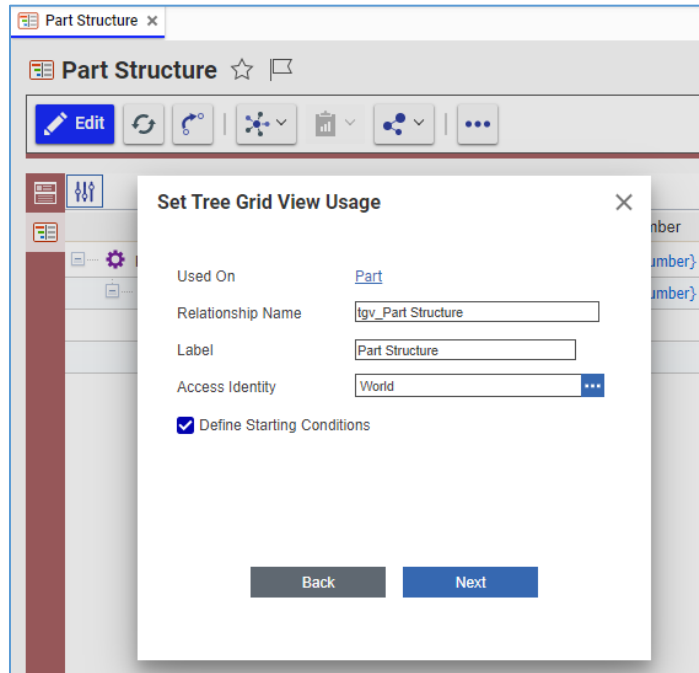


Figure 34.

5. Define the starting condition for query definition execution. The Starting condition is applied once to the root item of the query definition. It allows to get the resolved part structure for one part item.

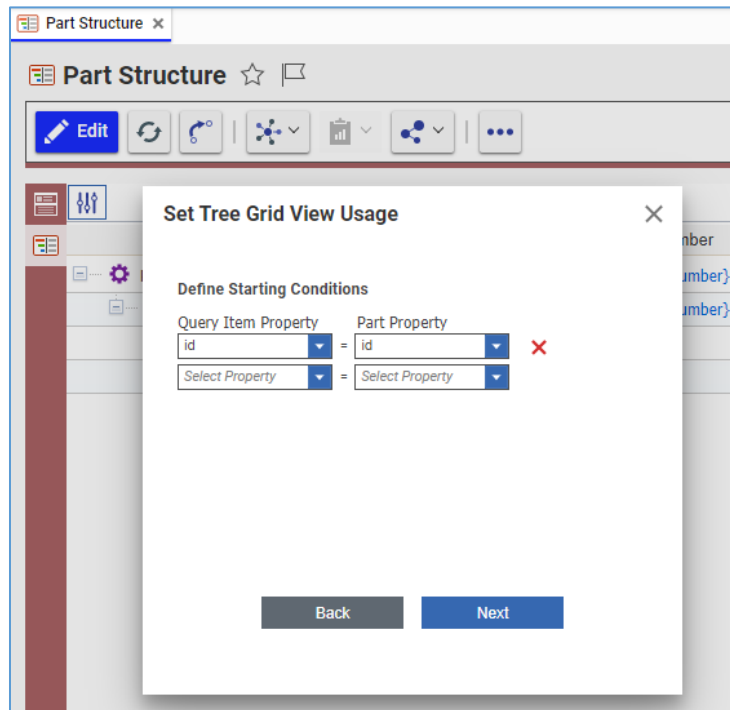


Figure 35.

6. Click **Next**. The following dialog box appears:

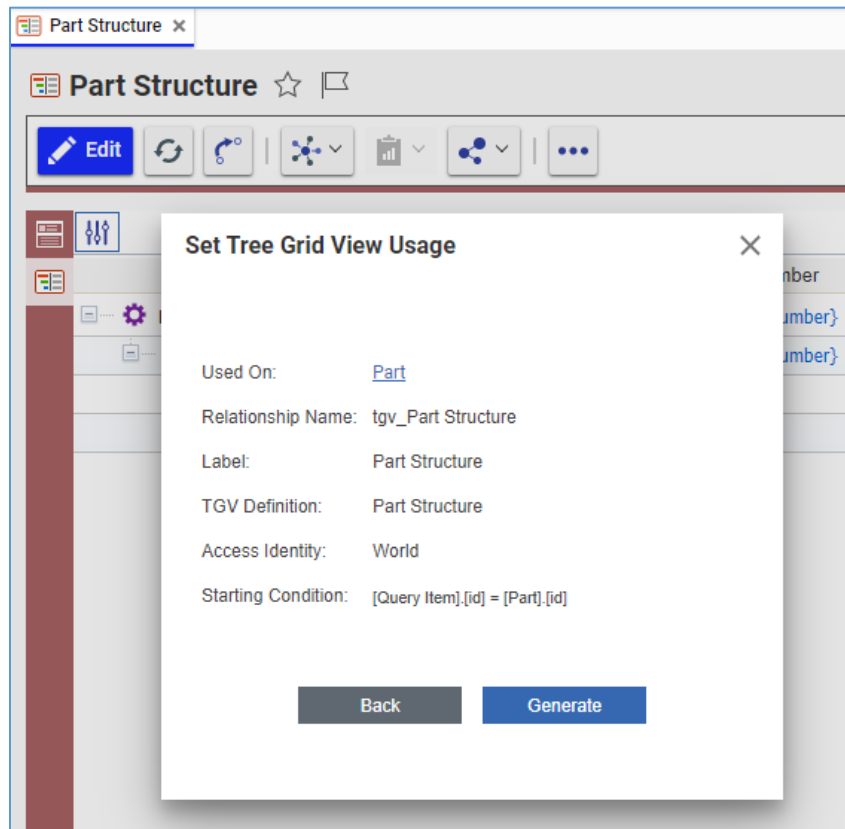


Figure 36.

7. Click **Generate** to create the relationship with the Tree Grid View.

6.3.3 Adding “Set Effectivity Criteria” Button to the Tree Grid View Toolbar

1. Create a Presentation Configuration Item and specify a name for it.
2. In the Command Bar Section relationships tab, select the `effs_tgvToolbarCustomization` Command Bar Section item. The `effs_tgvToolbarCustomization` Command Bar Section is provided out of the box and contains the “Set Effectivity Criteria” toolbar button which will be added to the Tree Grid View toolbar in the next steps.

- Specify a value for the “For Identity” cell and save the Presentation Configuration item.

effs guide configuration ☆

Save Done Discard

Presentation Configuration

Color

Name
effs guide configuration

Command Bar Section cui_PresentConfigWinSection Presentation Command

CommandBarSection ☆

Hidden

Classification	Name	Builder Meth...	Label	Description	Additional Da...	Location [...]	sort_order	For Identity [...]	For Classifica...
Data Model	effs_tgvToolbarCustomization					TGV_Toolbar	128	World	

Figure 37.

Note: If the Presentation Configurations category is not displayed in the TOC, you will need to expose the category by opening the Presentation Configuration ItemType and adding TOC Access for the current user.

- Open the Tree Grid View item where the “Set Effectivity Criteria” button will be added and specify the just created Presentation Configuration in the **Linked Toolbar/Context Menu** field.

Part Structure ☆

Save Done Discard

Tree Grid View

Name
Part Structure

Query Definition
[Part-Structure](#)

Context Item Type
[Part](#)

Description

Max Visible Children On Expand
100

Max Grow Levels
3

Auto Grow On Refresh
☒

Linked Toolbar/Context Menu
effs guide configuration

Figure 38.

- Click Save to add the “Set Effectivity Criteria” button to all Tree Grid View toolbars which use this Tree Grid View definition.

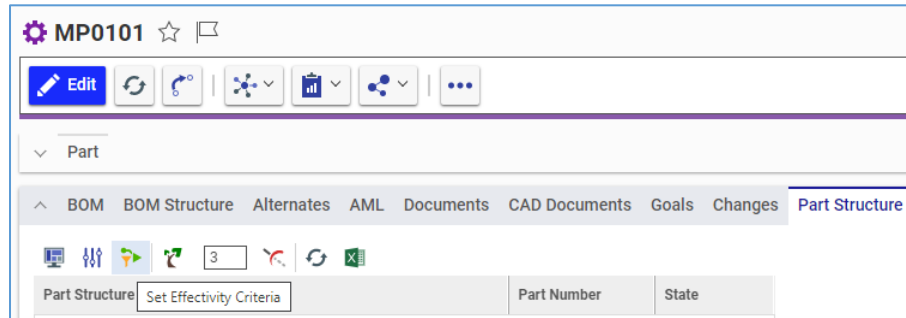


Figure 39.

6.3.4 Using Effectivity Criteria Filter Dialog for Resolving Effective Items

The resolved part structure for the current part item is displayed in the Tree Grid View. [Figure 40](#) shows the related parts. The Effectivity Expressions have not been added yet.

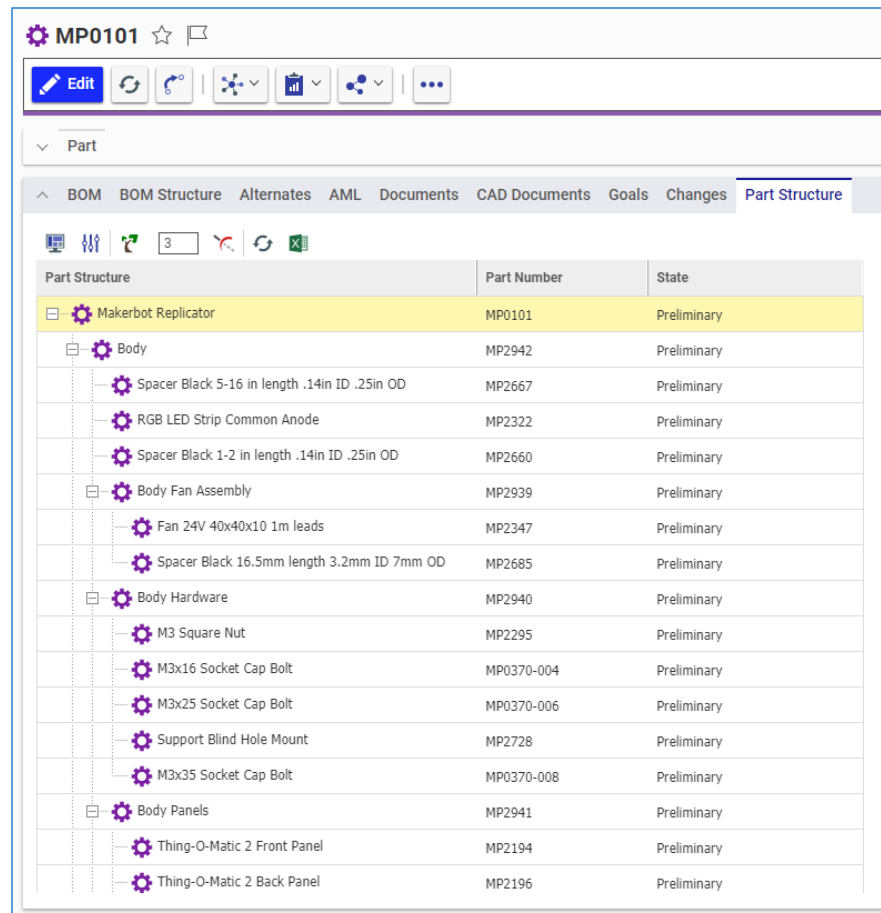


Figure 40.

6.3.4.1 Adding Effectivity Expressions to the Structure Relationships

In this example, Part “MP2940” has related Parts as shown in Figure 41. Effectivity Expressions will be added to the Part BOM relationships with the “MP0370-004” and “MP0370-006” related Part items.

1. Open the “MP2940” Part item which is present in the Part Structure Tree Grid View.
2. Click **Edit** so that the Part BOM relationships can be edited too.
3. Right click on the “MP0370-004” Part row and click **Open** to open the Part BOM relationship item.

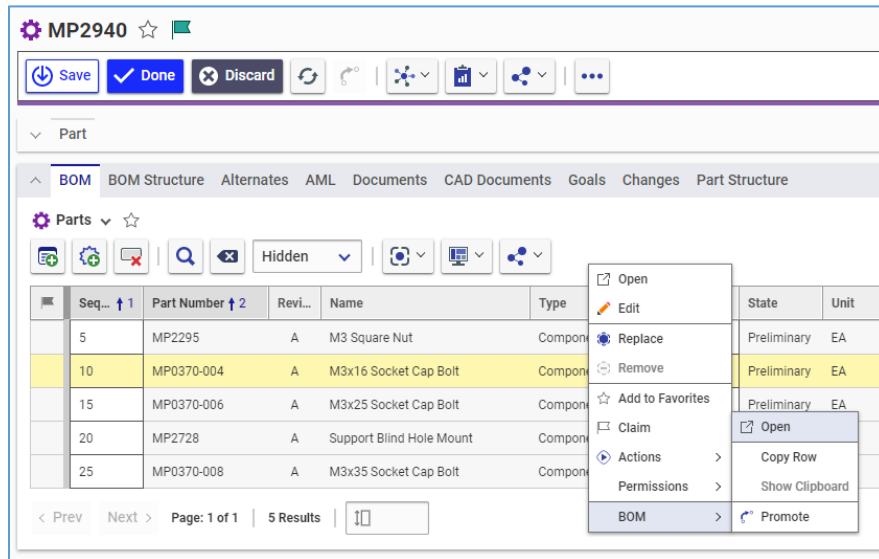


Figure 41.

4. Go to the “Effectivity” relationship tab and add a new Effectivity Expression. Set “3D Printer” Effectivity Scope in the Scope cell. Right click on the Effectivity Expression row and select **Open** to open the expression item.

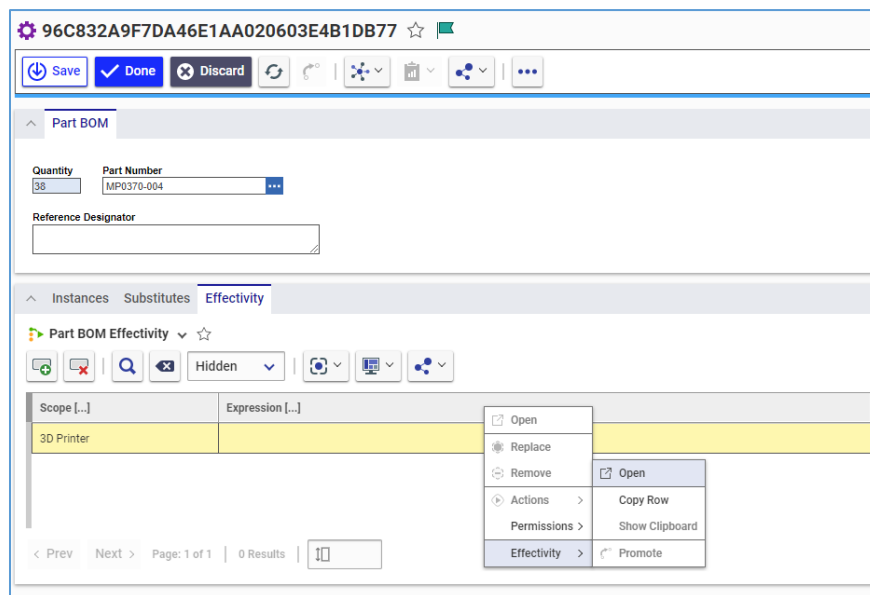


Figure 42.

5. Fill the **Effectivity Expression** field with the expression: "Model = Replicator and Unit >= 100".

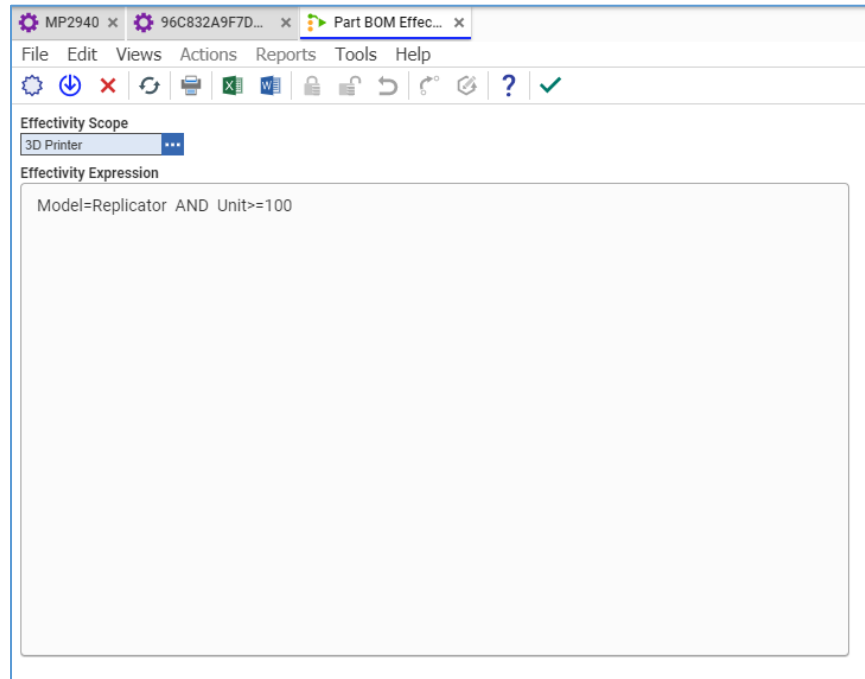


Figure 43.

6. Save, unlock, and close the Effectivity Expression item.
7. Save and close the Part BOM item

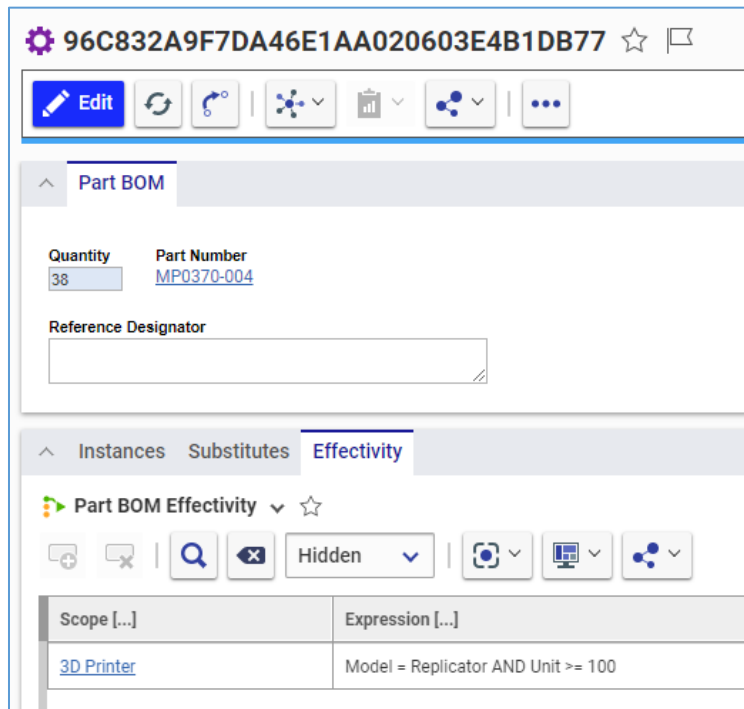


Figure 44.

8. Repeat steps 3-7 for the related Part “MP0370-006” and add two Effectivity Expressions: “Model = [Replicator 2]”, “[Production Date] >= [7/1/2021]”.

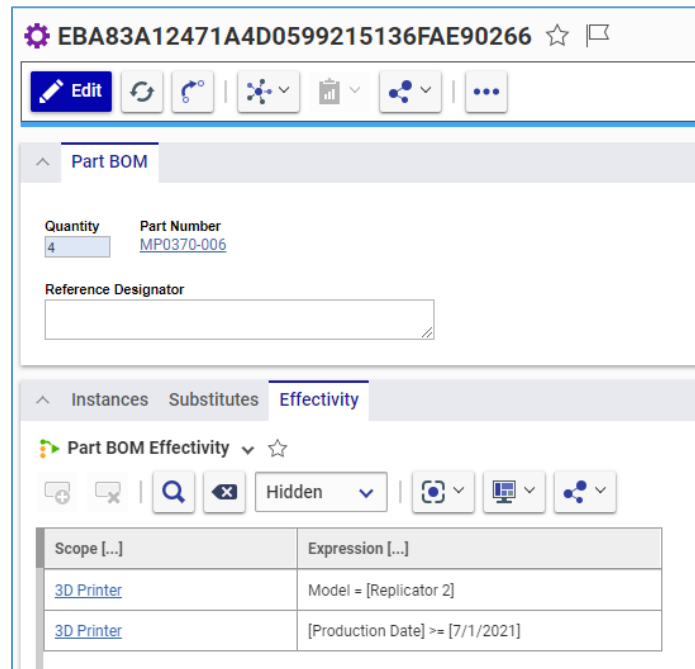


Figure 45.

6.3.4.2 Specifying Effectivity Criteria for Resolution

The Effectivity Criteria Filter dialog can be opened via the “Set Effectivity Criteria” button of the Tree Grid View toolbar.

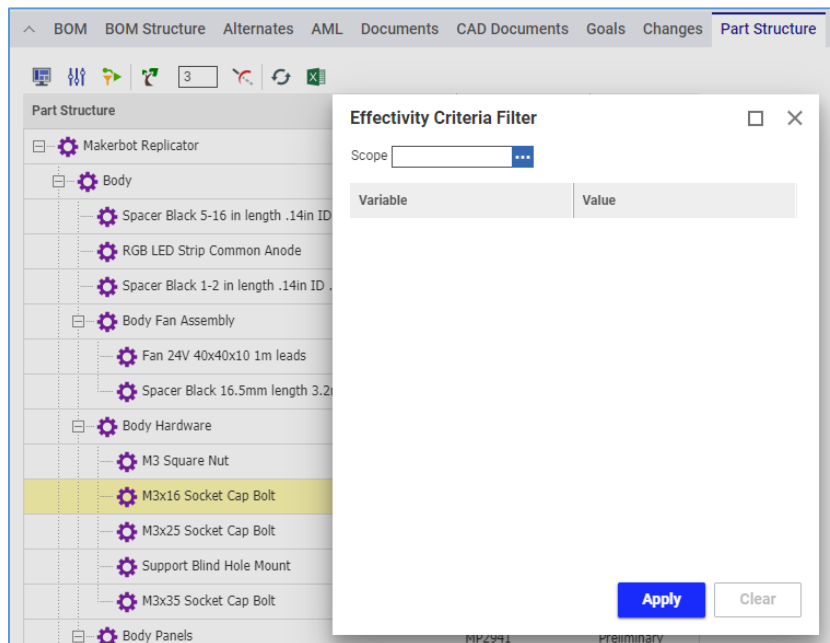
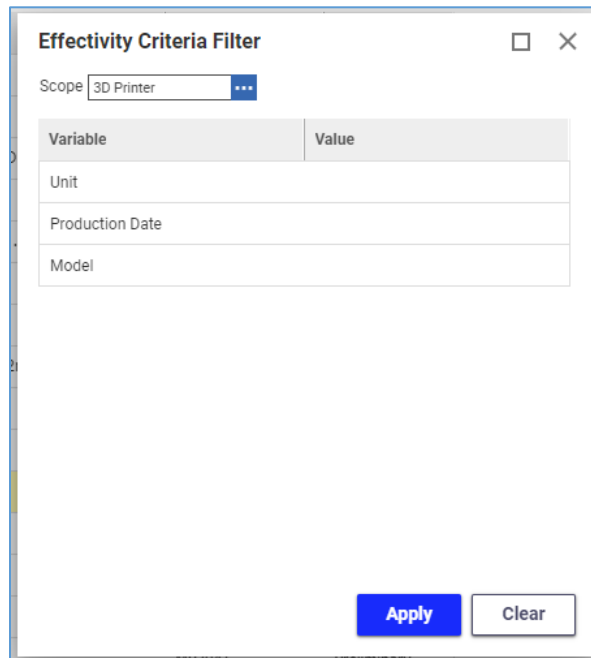


Figure 46.

The Effectivity Scope can be selected in the “Scope” field to populate the grid with Effectivity Variables which are present in the selected Scope.

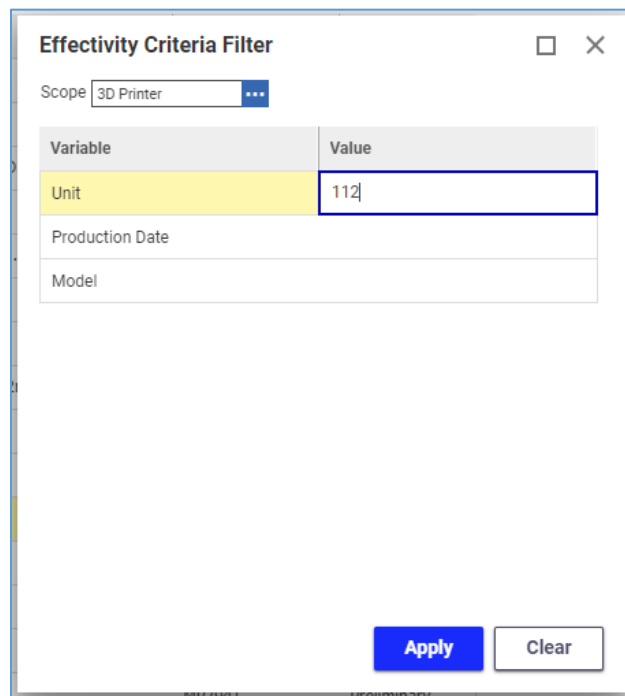


The dialog box titled "Effectivity Criteria Filter" has a "Scope" field set to "3D Printer". Below it is a table with two columns: "Variable" and "Value". The table contains three rows: "Unit", "Production Date", and "Model". At the bottom right are "Apply" and "Clear" buttons.

Variable	Value
Unit	
Production Date	
Model	

Figure 47.

For Integer and String Effectivity Variables, users can enter a variable value manually into the input field.



The dialog box titled "Effectivity Criteria Filter" has a "Scope" field set to "3D Printer". Below it is a table with two columns: "Variable" and "Value". The table contains three rows: "Unit", "Production Date", and "Model". The "Unit" row is highlighted in yellow, and the "Value" field for "Unit" contains the text "112". At the bottom right are "Apply" and "Clear" buttons.

Variable	Value
Unit	112
Production Date	
Model	

Figure 48.

For Date Effectivity Variables, users can enter values manually or via the Date Picker.

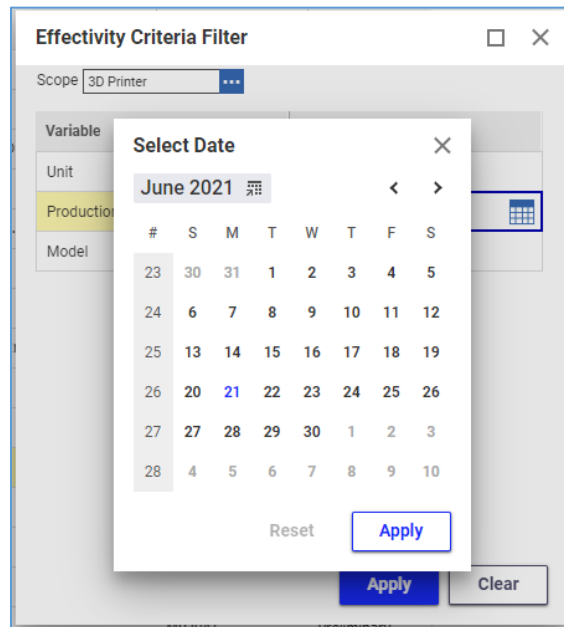


Figure 49.

For List and Item Effectivity Variables, users can enter values manually or via the dropdown list.

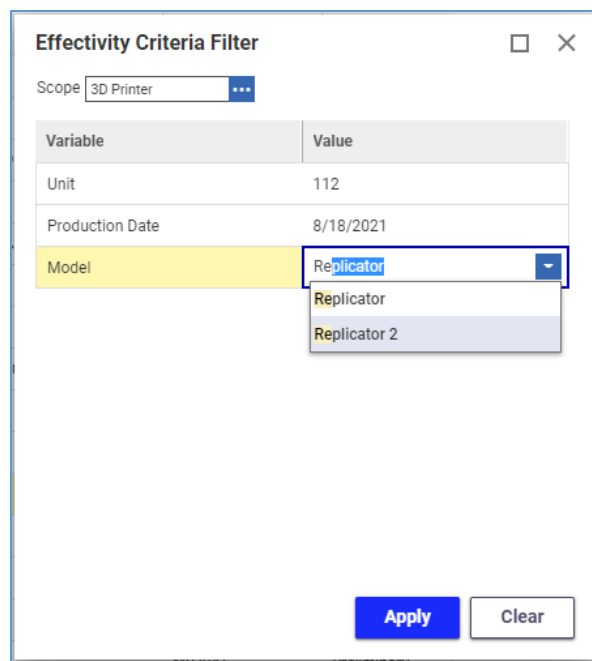
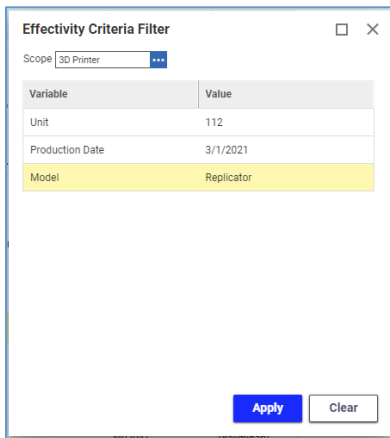


Figure 50.

The **Apply** button converts the entered Effectivity Variable values into Effectivity Criteria in the free-form Boolean expression format and refreshes the Tree Grid View with the item structure resolved according to the provided Effectivity Criteria.

For example, a user enters and applies the following Effectivity Criteria: Unit = 112, Production Date = 3/1/2021, Model = Replicator.



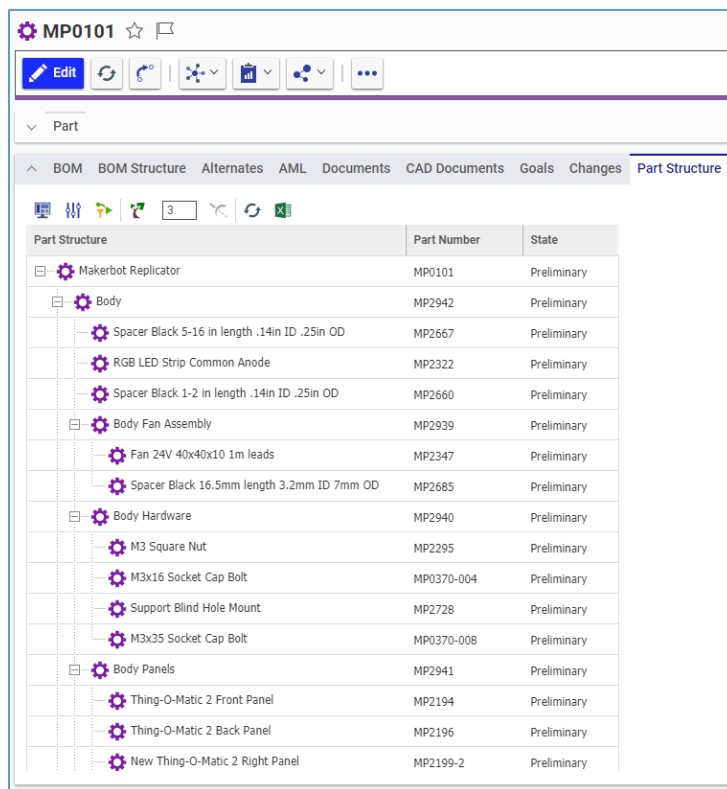
The dialog box titled "Effectivity Criteria Filter" has a "Scope" dropdown set to "3D Printer". It contains a table with the following data:

Variable	Value
Unit	112
Production Date	3/1/2021
Model	Replicator

At the bottom right, there are "Apply" and "Clear" buttons.

Figure 51.

In this case Part "MP0370-006" is no longer displayed in the Tree Grid View because its Effectivity Expressions "Model = [Replicator 2]", "[Production Date] >= [7/1/2021]" conflict with the provided Effectivity Criteria.

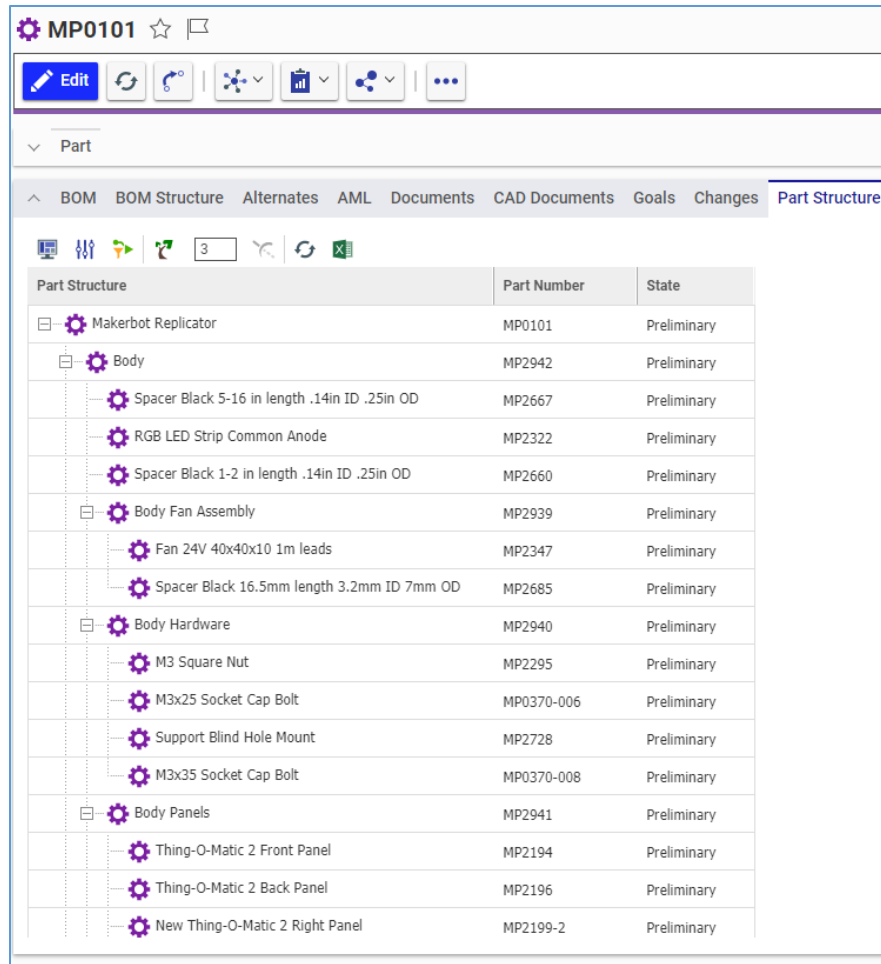


The screenshot shows the "MP0101" part structure. The "Part Structure" tab is active, displaying a table of components and their states.

Part Structure	Part Number	State
Makerbot Replicator	MP0101	Preliminary
Body	MP2942	Preliminary
Spacer Black 5-16 in length .14in ID .25in OD	MP2667	Preliminary
RGB LED Strip Common Anode	MP2322	Preliminary
Spacer Black 1-2 in length .14in ID .25in OD	MP2660	Preliminary
Body Fan Assembly	MP2939	Preliminary
Fan 24V 40x40x10 1m leads	MP2347	Preliminary
Spacer Black 16.5mm length 3.2mm ID 7mm OD	MP2685	Preliminary
Body Hardware	MP2940	Preliminary
M3 Square Nut	MP2295	Preliminary
M3x16 Socket Cap Bolt	MP0370-004	Preliminary
Support Blind Hole Mount	MP2728	Preliminary
M3x35 Socket Cap Bolt	MP0370-008	Preliminary
Body Panels	MP2941	Preliminary
Thing-O-Matic 2 Front Panel	MP2194	Preliminary
Thing-O-Matic 2 Back Panel	MP2196	Preliminary
New Thing-O-Matic 2 Right Panel	MP2199-2	Preliminary

Figure 52.

If Effectivity Criteria is changed to Unit = 112, Production Date = 3/1/2021, Model = [Replicator 2] then Part "MP0370-004" is not displayed because its Effectivity Expression "Model = Replicator and Unit >= 100" conflicts with provided Effectivity Criteria.



Part Structure	Part Number	State
Makerbot Replicator	MP0101	Preliminary
Body	MP2942	Preliminary
Spacer Black 5-16 in length .14in ID .25in OD	MP2667	Preliminary
RGB LED Strip Common Anode	MP2322	Preliminary
Spacer Black 1-2 in length .14in ID .25in OD	MP2660	Preliminary
Body Fan Assembly	MP2939	Preliminary
Fan 24V 40x40x10 1m leads	MP2347	Preliminary
Spacer Black 16.5mm length 3.2mm ID 7mm OD	MP2685	Preliminary
Body Hardware	MP2940	Preliminary
M3 Square Nut	MP2295	Preliminary
M3x25 Socket Cap Bolt	MP0370-006	Preliminary
Support Blind Hole Mount	MP2728	Preliminary
M3x35 Socket Cap Bolt	MP0370-008	Preliminary
Body Panels	MP2941	Preliminary
Thing-O-Matic 2 Front Panel	MP2194	Preliminary
Thing-O-Matic 2 Back Panel	MP2196	Preliminary
New Thing-O-Matic 2 Right Panel	MP2199-2	Preliminary

Figure 53.

If the Effectivity Criteria is changed to Unit = 7, Production Date = 3/1/2021, Model = Replicator then both "MP0370-004" and "MP0370-006" Parts are not displayed because their Effectivity Expressions conflict with the provided Effectivity Criteria.

Part Structure			Part Number	State
[-]	Makerbot Replicator		MP0101	Preliminary
[-]	Body		MP2942	Preliminary
		Spacer Black 5-16 in length .14in ID .25in OD	MP2667	Preliminary
		RGB LED Strip Common Anode	MP2322	Preliminary
		Spacer Black 1-2 in length .14in ID .25in OD	MP2660	Preliminary
[-]	Body Fan Assembly		MP2939	Preliminary
		Fan 24V 40x40x10 1m leads	MP2347	Preliminary
		Spacer Black 16.5mm length 3.2mm ID 7mm OD	MP2685	Preliminary
[-]	Body Hardware		MP2940	Preliminary
		M3 Square Nut	MP2295	Preliminary
		Support Blind Hole Mount	MP2728	Preliminary
		M3x35 Socket Cap Bolt	MP0370-008	Preliminary
[-]	Body Panels		MP2941	Preliminary
		Thing-O-Matic 2 Front Panel	MP2194	Preliminary
		Thing-O-Matic 2 Back Panel	MP2196	Preliminary
		New Thing-O-Matic 2 Right Panel	MP2199-2	Preliminary
		Thing-O-Matic 2 Top Panel	MP2193	Preliminary

Figure 54.

When the user clicks the “Set Effectivity Criteria” button again, the previously applied Effectivity Criteria will be displayed in the Effectivity Criteria Filter dialog. Each Effectivity Variable value can be cleared separately by removing the cell value. Alternatively, the **Clear** button can be used to clear all Effectivity Variable values and the selected Scope.

After the Effectivity Criteria is cleared and **Apply** is clicked, an unfiltered BOM is displayed including both Parts.

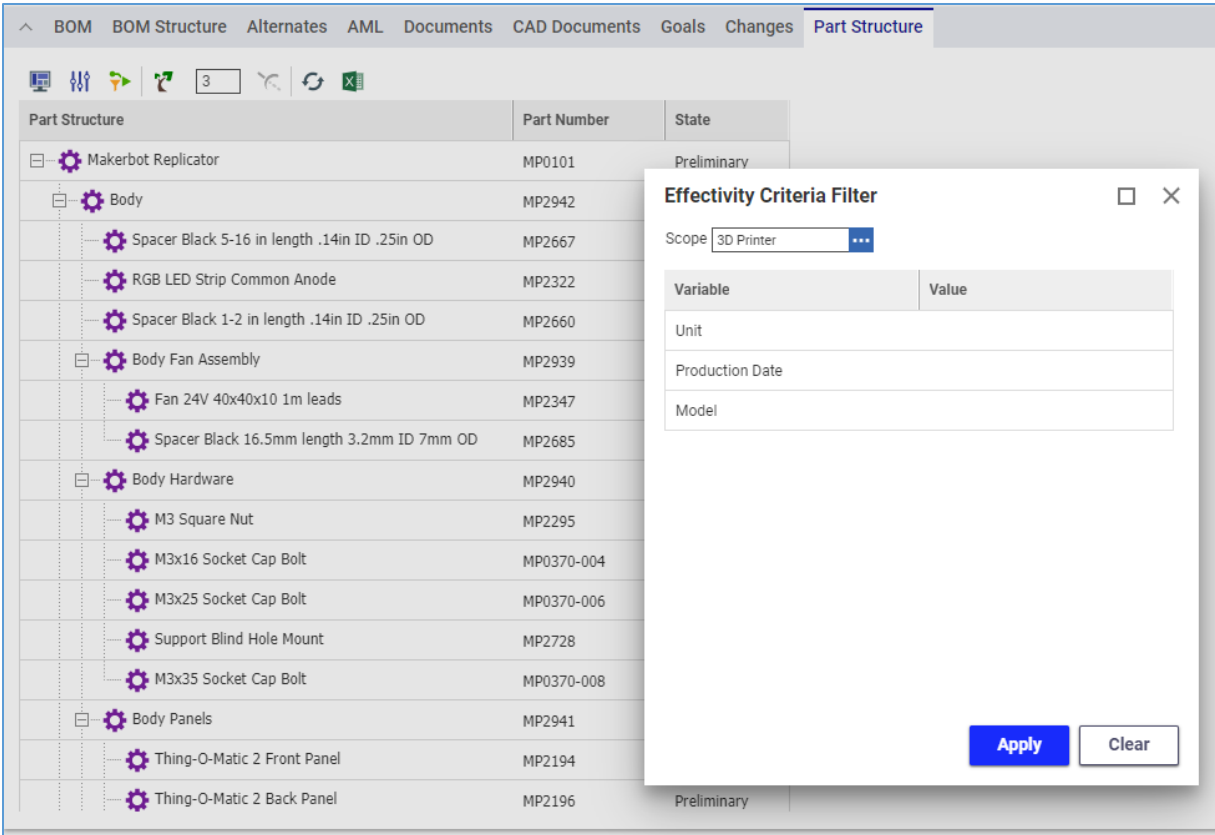


Figure 55.

7 Dynamic Tree Grid Control API

The Dynamic Tree Grid Control is available in Effectivity Services. This section describes the API. The Dynamic Tree Grid control is inherited from the TreeGrid component and has all its public methods, fields, and events.

7.1 Constructor

Table 5 describes two parameters that the constructor supports.

Table 4: Constructor-supported parameters

Name	Type	Description
dom	Object	Required parameter. It is a DOM element that will be used as a control container.
settings	Object	Optional parameter. It contains the following global settings for grid initialization: multiSelect – gives you the ability to select multiple rows in the grid. The default value is false. editable – determines whether a grid can be edited or not. The default value is true. sortable – determines whether grid columns can be sorted. The default value is true. draggableColumns – determines whether grid columns can be moved. The default value is true. resizable – determines whether columns can be resized. The default value is true. If you do not specify the settings parameter, its default value will be used for grid initialization.

7.2 Public fields

DynamicTreeGrid does not have its own public fields. It uses the base TreeGrid component public fields.

7.3 Events

You can add and remove Events using the “on” and “off” public methods.

Each event listener callback accepts a single parameter: an event object of the type “CustomEvent” containing a “detail” property with specific event information.

Note: Column and row indexes are zero-based. Row indexing starts from the first row displayed in a grid and ends at the bottom row of the entire grid. It means that rows whose parents are collapsed/invisible are not counted. The column indexing starts from the first column on the left. All invisible rows and columns have an index of -1.

7.3.1 addRow

Event fired after each row is added to the grid when adding rows using the “addRows” or “loadData” methods.

Table 5: Property details for addRow

Name	Type	Description
rowId	String	ID of the added row.
parentId	String	ID of the parent row. Null if added row is root.

7.3.2 addRows

Event fired after all rows are added using the “addRows” or “loadData” methods.

Table 6: Property details for addRows

Name	Type	Description
rowIds	Array	Added row IDs.
parentId	String	ID of the parent row. The ID is Null if the added row is root.

7.3.3 removeRow

Event fired after deleting a row from the grid using the “removeRow” method.

Table 7: Property details for removeRow

Name	Type	Description
rowIds	Array	IDs of the deleted rows. Array contains ID of the deleted row and IDs of all descendant rows.
parentId	String	ID of the parent row. The ID is Null if the deleted row is a root row.

7.3.4 removeAllRows

Event fired after deleting all rows from the grid using the “removeAllRows” method.

Table 8: Property details for removeRows

Name	Type	Description
rowIds	Array	IDs of the deleted rows.

7.3.5 addColumn

Event fired after adding a new column using the “addColumn” or “loadData” methods.

Table 9: Property details for addColumn

Name	Type	Description
columnName	String	Column name.

7.3.6 removeColumn

Event fired after deleting a column using the “removeColumn” method.

Table 10: Property details for removeColumn

Name	Type	Description
columnName	String	Column name.

7.4 Common objects description

This section describes common objects frequently used in the DynamicTreeGrid public methods.

7.4.1 Metadata objects

A Metadata object is an object that contains additional information about a cell that is required for the grid editors and formatters to render the cell properly.

A Metadata object can have the following properties:

- **formatter** – optional property.

The formatter name used to display cell content. If you do not specify a “formatter” property, the formatter type is determined by the grid based on the cell value. E.g., if the cell value is a string the ‘text’ formatter will be used, and if the cell value is boolean the ‘boolean’ formatter will be used.

- **editor** – optional property.

The name of the editor used to display cell content when the cell is in the edit state. If you do not specify an “editor” property, the editor type will be determined by the grid.

- Other properties that can be used in the formatter and editor functions from the “metadata” parameter.

Example:

```
{
  formatter: 'select',
  editor: 'select',
  options: [
    {label: 'Red', value: 'id1'},
    {label: 'Green', value: 'id2'},
    {label: 'Blue', value: 'id3'}
  ]
}
```

7.4.2 Column settings object

The Column settings object contains additional settings applicable to the corresponding column.

If a grid is not editable/resizable/sortable, then you cannot edit/resize/sort columns. However, if the grid is editable/resizable/sortable you can adjust these columns. The Column setting does not change when the corresponding grid setting is changed.

The Column settings object can have the following optional properties:

- **resizable** enables you to decide whether to resize a column. The default value is obtained from grid settings when adding a new column.
- **editable** enables you to decide whether to edit cells in this column. The default value is obtained from grid settings when adding a new column. This setting value will be used as a default value for the cell's editability setting when adding new rows. You can also use the “setCellEditability” and “getCellEditability” methods to set/get cell editability.
- **sortable** enables you to decide whether to sort by this column. The default value is obtained from the grid settings when adding a new column.

- **visible** enables you to decide if the column should be visible or not. The Default value is `true` when adding a new column.

Example:

```
{
    resizable: false,
    sortable: true,
    editable: false,
    visible: true
}
```

7.4.3 Column object

The Column object is used in the “loadData” and “addColumn” methods. It can contain the following properties:

- **name** – required property. Column name.
- **label** – optional property. The column name is used as the label if the “label” property is not set.
- **width** – optional property. The width is calculated automatically based on the column label if the “width” property is not set.
- **metadata** – optional property. It contains a metadata object for all cells in this column. Column metadata is used only for cells without their own metadata.
- **settings** – optional property containing settings that are applied to the corresponding column.

Example:

```
{
    label: 'Column 1',
    width: 100,
    name: 'property1',
    metadata: {
        formatter: 'boolean'
    },
    settings: {
        resizable: false,
        editable: false
    }
}
```


7.4.4 Row object

The Row object is used in the “loadData” and “addRows” methods. It can contain different properties. If the row object property name matches the column name, then its value is used as a cell value. The Row object property is updated automatically when the corresponding grid cell value is changed.

Example:

```
{
  property1: 'value 1',
  property2: 'value 2',
  someOtherProperties: 'additional property'
}
```

7.5 Public methods

7.5.1 loadData

Initializes the grid with specified rows and columns.

If the “columns” parameter is not specified, the columns will be obtained automatically from the “rows” parameter. Each unique property of the row object will represent a column with the same name.

If both the “rows” and “columns” parameters are not specified, the method removes all existing rows and columns.

Note: The grid automatically updates the corresponding properties of the provided row object when the cell value is changed.

Table 11: Optional Input parameters for loadData

Name	Type	Description
rows	Array	An array containing the row objects described in section 10.4.4 . If the parameter is not an array, the rows are not added.
columns	Array	An array containing column objects described in section 10.4.3 . The Array's element position determines the column order.

Return value

Array – an array that contains the IDs of the added rows.

Example of the “rows” array parameter:

```
[
  {
    property1: 'row 1',
    property2: 'value2',
    someOtherProperties: 'additional property'
  },
  {
    property1: 'row 2'
  }
]
```

Example of the “columns” array parameter:

```
[
  {
    label: 'Column 1',
    width: 55,
    name: 'property1',
    metadata: {
      formatter: 'select',
      editor: 'select',
      options: [
        {label: 'Red', value: 'id1'},
        {label: 'Green', value: 'id2'},
        {label: 'Blue', value: 'id3'}
      ]
    }
  },
  {
    name: 'property2',
    settings: {
      resizable: false,
      sortable: true,
      editable: false
    }
  }
]
```

7.5.2 obtainRowID

obtainRowID is an overridable handler which should return a unique row id for the given row object when adding new rows using the “loadData” or “addRows” methods. It returns null by default. The Unique row id will be generated automatically if the handler returns a falsy value, e.g., false, empty string, null or undefined. If the handler returns a truthy value which is not a string or grid that already contains a row with the same id, an error will be thrown in the “loadData” or “addRows” methods.

Table 12: Input parameters for obtainRowId

Name	Type	Description
rowInfo	Object	Row object which is used to add a new row.
parentId	String	Parent row ID. The ID will be Null if the row is added as root.

Return value

String—row ID.

Example:

```
dynamicTreeGrid.obtainRowId = function(rowObj, parentId) {
    return rowObj.id_property;
};
```

7.5.3 addRows

Adds rows to the specified position in the grid.

Table 13: Input parameters for addRows

Name	Type	Description
rows	Array	Required parameter. Arrays containing row objects are described in section 10.4.4 . If the parameter is not an array, the rows will not be added.
parentId	String	Optional parameter. ID of the parent row. If the “parentId” parameter is null or undefined, rows will be added as roots.
position	Number	Optional parameter. It is a zero-based index that is used to insert new rows into the child row IDs array of the parent row (or roots array if parentId is not specified). The Index can be greater than or equal to 0 and less than or equal to the children array length. If the “position” parameter is not specified, new child rows will be appended to the end of the children array.

Name	Type	Description
		The getChildRowIds method can be used to find out the row index in the children array.
doRender	Boolean	Optional parameter. Renders grid after adding rows. The default value is true.

Return value

Array—array which contains IDs of the added rows.

7.5.4 removeRow

Removes the row with the specified ID from the grid and grid rows collection.

Table 14: Input parameters for removeRows

Name	Type	Description
rowId	String	Row ID.

7.5.5 removeAllRows

Removes all rows from the grid and the grid rows collection.

7.5.6 selectRow

Selects the row with the specified ID. Otherwise, the specified row becomes selected, and all previously selected rows stay selected.

Table 15: Input parameters for selectRow

Name	Type	Description
rowId	String	Row ID.

7.5.7 deselectRow

Cancels the selection of the specified row.

Table 16: Input parameters for deselectRow

Name	Type	Description
rowId	String	Row ID.

7.5.8 addColumn

Adds a column to the grid.

Table 17: Input parameters for addColumn

Name	Type	Description
column	Object	Required parameter. The Column object is described in the section 10.4.3 .
position	Number	Optional parameter. Zero based column index. If its position is not specified, the column will be appended after existing columns.

7.5.9 removeColumn

Removes the specified column from the grid and the grid columns collection.

Table 18: Input parameters for removeColumn

Name	Type	Description
name	String	Column name.

7.5.10 getChildRowIds

Returns an Array containing the child row IDs for the specified parent row. This method returns the IDs of the root rows if the "parentId" parameter is null or undefined.

Table 19: Input parameters for getChildRowIds

Name	Type	Description
parentId	String	Optional parameter. ID of the parent row.

Return value

Array—array with child row IDs or an empty array if there are no children.

7.5.11 getRowUserData

Gets user data stored by the specified row and key.

Table 20: Input parameters for getRowUserData

Name	Type	Description
rowId	String	Row ID.
key	String	Key for storing user data.

Return value

Anything—user data stored by the specified row and key.

Example:

```
let itemtype = grid.getRowUserData(rowId, 'itemtype');
```

7.5.12 setRowUserData

Sets user data for the specified row and key.

Table 21: Input parameters for setRowUserData

Name	Type	Description
rowId	String	Row ID
key	String	Key for storing user data
value	Anything	Value

Example:

```
grid.setRowUserData(rowId, 'itemtype', 'Part');
```

7.5.13 setCellEditability

Sets cell edit availability.

Table 22: Input parameters for setCellEditability

Name	Type	Description
rowId	String	Required parameter. Row ID.
columnName	String	Required parameter. Column name.
editable	Boolean	Optional parameter. Can a cell be edited or not? Default value is true.

7.5.14 getCellEditability

Returns `true` if the specified cell is editable, otherwise it returns `false`.

Table 23: Input parameters for getCellEditability

Name	Type	Description
rowId	String	Row ID.
columnName	String	Column name.

Return value

Boolean—is cell editable or not.

7.5.15 setCellValue

Sets cell value and updates the corresponding property in the user row object.

Table 24: Input parameters for setCellValue

Name	Type	Description
rowId	String	Row ID
columnName	String	Column name
value	Anything	Cell value

7.5.16 getCellValue

Gets the cell value.

Table 25: Input parameters for getCellValue

Name	Type	Description
rowId	String	Row ID
columnName	String	Column name

Return value

Anything—cell value.

7.5.17 getColumnCount

Gets a count of all columns (visible and invisible).

Return value

Number

7.5.18 getVisibleColumnCount

Gets a count of visible columns.

Return value

Number

7.5.19 getColumnIndex

Gets a column index by the column name. Returns -1 if the column is not found or is invisible.

Table 26: Input parameters for getColumnIndex

Name	Type	Description
columnName	String	Column name.

Return value

Number—column index

7.5.20 getColumnName

Gets the column name using the column index.

Table 27: Input parameters for getColumnName

Name	Type	Description
columnIndex	Number	Column index

Return value

String—column name

7.5.21 getColumnOrder

Gets Array with the names of visible columns in the same order as they are displayed in the grid.

Return value

Array

7.5.22 getAllColumnNames

Gets Array with the names of all columns in the order they are added to the grid.

Return value

Array

7.5.23 getParentId

Gets the ID of the parent row for the specified row. Returns null if the specified row is root and has no parent.

Table 28: Input parameters for getParentId

Name	Type	Description
rowId	String	Row ID.

Return value

String—parent row ID.

7.5.24 containsRow

Returns `true` if the grid has a row with the specified ID in the rows collection, otherwise it returns `false`.

Table 29: Input parameters for containsRow

Name	Type	Description
rowId	String	Row ID.

Return value

Boolean

7.5.25 containsColumn

Returns `true` if the grid has a column with the specified name in the columns collection, otherwise it returns `false`.

Table 30: Input parameters for containsColumn

Name	Type	Description
columnName	String	Column name.

Return value

Boolean

7.5.26 expandAll

Expands all rows with their descendants.

7.5.27 collapseAll

Collapses all rows with their descendants.

7.5.28 isRowExpanded

Returns `true` if the specified row is expanded, otherwise it returns `false`.

Table 31: Input parameters for isRowExpanded

Name	Type	Description
rowId	String	Row ID.

Return value

Boolean

7.5.29 isRowVisible

Returns `true` if the specified row is visible, otherwise it returns `false`.

Table 32: Input parameters for isRowVisible

Name	Type	Description
rowId	String	Row ID.

Return value

Boolean

7.5.30 getVisibleRowCount

Gets the number of visible rows that can be displayed in the grid.

Note: The number of visible rows is not the total rows count. For example, if the grid has only one root row with 10 children and the root row is collapsed, the `getVisibleRowCount()` method will return 1.

Return value

Number

7.5.31 getAllRowCount

Gets the number of all rows (visible and invisible) contained in the rows collection.

Return value

Number

7.5.32 getVisibleRowIds

Gets the Array with the IDs of visible rows displayed in the grid.

Return value

Array

7.5.33 getAllRowIds

Gets the Array with the IDs of all rows contained in the rows collection.

Return value

Array

7.5.34 getRowId

Gets the row ID by the row index.

Table 33: Input parameters for getRowId

Name	Type	Description
rowIndex	Number	Row index (zero based, from "top" to "bottom").

Return value

String—row ID.

7.5.35 getRowIndex

Gets the row index using the row ID. The method returns -1 if the row is not found or not visible.

Table 34: Input parameters for getRowIndex

Name	Type	Description
rowId	String	Row ID

Return value

Number—row index.

7.5.36 moveColumn

Moves a column to the specified position.

Table 35: Input parameters for moveColumn

Name	Type	Description
columnName	String	Column name
columnIndex	Number	New column position – zero based index.

7.5.37 setColumnSettings

Sets settings for the specified column.

Table 36: Input parameters for setColumnSettings

Name	Type	Description
columnName	String	Column name
settings	Object	The Column settings object described in the section 10.4.2 Note: If a column setting is not specified in the column object, its value will not be changed.

7.5.38 getColumnSettings

Gets settings for the specified column.

Table 37: Input parameters for getColumnSettings

Name	Type	Description
columnName	String	Column name.

Return value

Object—column settings object described in section [10.4.2](#).

7.5.39 setColumnMetadata

Sets metadata for the specified column that will be used by the column cells without their own metadata.

Table 38: Input parameters for setColumnMetadata

Name	Type	Description
columnName	String	Column name
metadata	Object	Metadata object described in the section 10.4.1 .

7.5.40 getColumnMetadata

Gets metadata for the specified column. Returns null if the column has no metadata.

Table 39: Input parameters for getColumnMetadata

Name	Type	Description
columnName	String	Column name

Return value

Object—metadata object described in section [10.4.1](#).

7.5.41 setColumnVisibility

Sets column visibility.

Table 40: Input parameters for setColumnVisibility

Name	Type	Description
columnName	String	Required parameter. Column name.
visible	Boolean	Optional parameter. Column visibility. The default value is true.

7.5.42 getColumnVisibility

Returns `true` if the specified column is visible, otherwise it returns `false`.

Table 41: Input parameters for getColumnVisibility

Name	Type	Description
columnName	String	Column name

Return value

Boolean

7.5.43 setColumnLabel

Sets the label for the specified column.

Table 42: Input parameters for setColumnLabel

Name	Type	Description
columnName	String	Column name
label	String	Column label.

7.5.44 getColumnLabel

Gets the label for the specified column.

Table 43: Input parameters for getColumnLabel

Name	Type	Description
columnName	String	Column name

Return value

String—column label.

7.5.45 setColumnWidth

Sets the width for the specified column.

Table 44: Input parameters for setColumnWidth

Name	Type	Description
columnName	String	Column name
width	Number	Column width

7.5.46 getColumnWidth

Gets the width for the specified column.

Table 45: Input parameters for getColumnWidth

Name	Type	Description
columnName	String	Column name

Return value

Number—column width.

7.5.47 setCellMetadata

Sets metadata for the specified cell.

Table 46: Input parameters for setCellMetadata

Name	Type	Description
rowId	String	Row ID
columnName	String	Column name
metadata	Object	Metadata object described in the section 10.4.1

7.5.48 getCellMetadataOnly

Gets metadata for the specified cell. Returns null if the cell has no metadata (even if the column has metadata).

Table 47: Input parameters for getCellMetadataOnly

Name	Type	Description
rowId	String	Row ID
columnName	String	Column name

Return value

Object—metadata object described in section [10.4.1](#).

8 Appendix

8.1 How-to Example - Setting a default Effectivity Scope in “Effectivity Criteria Filter” dialog

This section discusses an approach to set a default Effectivity Scope on the Effectivity Criteria dialog with the corresponding variables based on the default scope.

There can be a variety of business rules to determine the default effectivity scope, such as item classification, item state, user identities, and teams. It may be necessary to pre-populate the Effectivity scope only if a single effectivity scope can be determined. In addition, business-specific logic may change over time.

It is ideal to have a single place where all default rule logic is defined/implemented and retrieved. A server-side method – with parameters as needed – will be an entry point for querying a default effectivity scope. The results can be cached across many requests. This server method can be called from either the client or server-side as shown in the following figure.

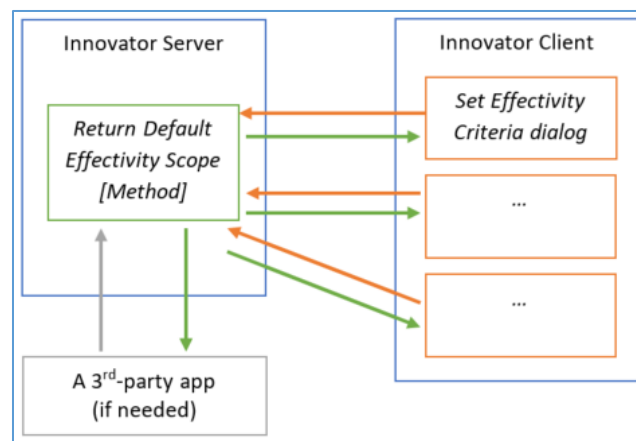


Figure 56.

For the new server-side method to identify a default effectivity scope:

- A new server-side method is created and given a name such as `effs_getScopeDefaults`. This name is abstract enough to expand the method's functionality as needed. For example, the method can return not only a default scope, but also default values for variables in that scope.
- The method can return a JSON string. The format may look like:

```
{
  "scope": {
    "id": "32-hex-id"
  }
}
```

The value of the `scope.id` property is the item ID of the effectivity scope item (`effs_scope` ItemType).

This format provides flexibility to consider future improvements with minimal to no changes.

- The method may accept parameters, as needed, for processing of the business logic.

As for the client-side changes to the Effectivity Criteria dialog, the current order of events is outlined as shown in Figure 57.

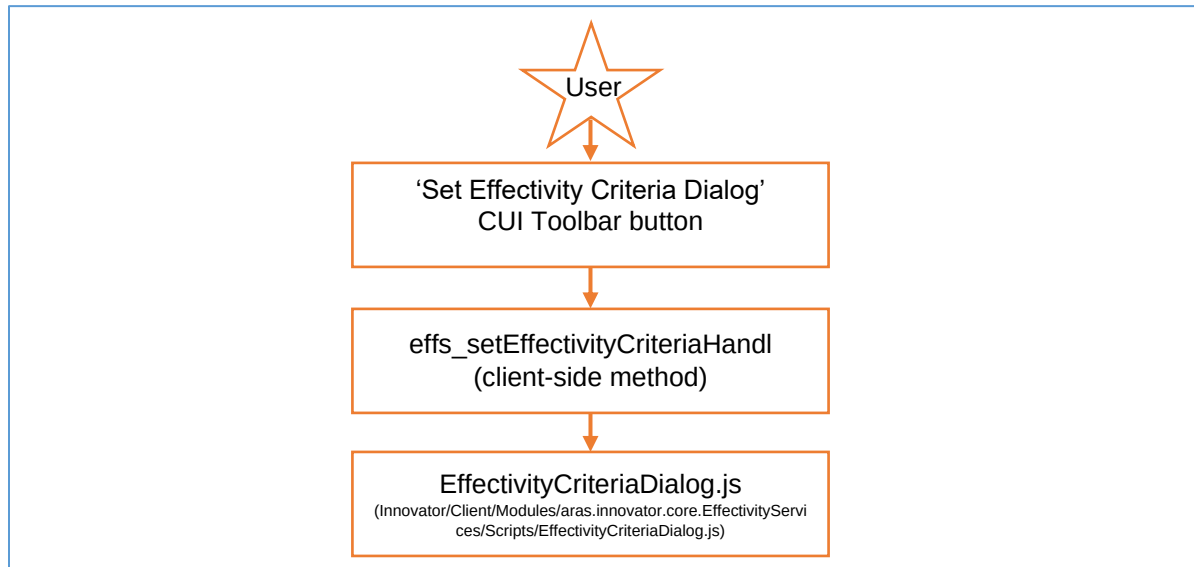


Figure 57.

Requesting the default scope from the server and passing that information along to the Effectivity Criteria dialog to be displayed requires the following modification:

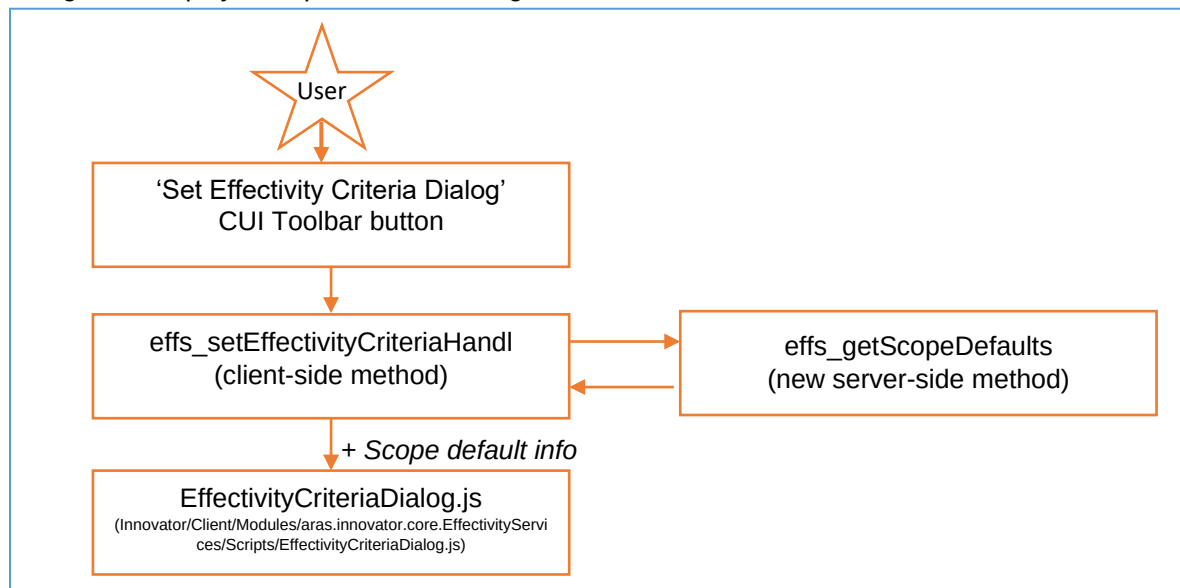


Figure 58.

The changes outlined in the previous figure can be accomplished with the following:

- In the `effs_setEffectivityCriteriaHandle` client-side method, make a request to the new server-side method to get default scope details. A client-side request to a server-side method can be achieved using `aras.applyMethod('effs_getScopeDefaults','')`;
- Construct an object from the response string that would be expected in the dialog `EffectivityCriteriaDialog`'s constructor (modifications in the constructor would be a next step). You can create an object from a string using `JSON.parse()`.
- Take the 'scope' property from the just constructed object and pass it as a new additional argument to `EffectivityCriteriaDialog`'s object creation in the `effs_setEffectivityCriteriaHandle` client-side method.
- After the changes from the steps outlined above, the `effs_setEffectivityCriteriaHandle` client-side method will look similar to the following:

```

42  if (!contextParameters.effectivityCriteriaDialog) {
43      /* START: CHANGES FOR DEFAULTING SCOPE */
44      const scopeDefaultsStringResponse = aras.applyMethod('effs_getScopeDefaults','');
45      const scopeDefaults = JSON.parse(scopeDefaultsStringResponse);
46      contextParameters.effectivityCriteriaDialog = new EffectivityCriteriaDialog(window.aras, scopeDefaults.scope);
47      /* END: CHANGES FOR DEFAULTING SCOPE */
48  }
49
50  contextParameters.effectivityResolutionConfigurationQueryDefinitionId = aras.evalMethod('effs_getResolutionCfgQdIdByTgvId', '', {
51      treeGridViewDefinitionId: contextParameters.tgvContext.getViewDefinitionId()
52  });

```

Figure 59.

- Open the following file:
Innovator/Client/Modules/aras.innovator.core.EffectivityServices/Scripts/EffectivityCriteriaDialog.js

Update the `EffectivityCriteriaDialog`'s constructor to process the new parameter. To pre-populate the scope field on the dialog form, add the `this.scope.id` property in the constructor. The resulting constructor may look like the following:

```

5      scope: null,
6
7      constructor: function (aras, scopeDefaults) {
8          this.aras = aras;
9          this.scope = scopeDefaults;
10     },

```

Figure 60.

8.2 How-to Example - Executing Query Definitions with Effectivity Resolution

This section outlines executing a Query Definition and applying Effectivity Resolution. This involves:

1. Retrieving the Query Definition AML
2. Specifying values for the query parameters of the Query Definition
3. Executing the Query Definition with Effectivity Resolution

8.2.1 Retrieve Query Definition AML

You can retrieve the Query Definition AML using the following AML query:

```
<Item type="qry_QueryDefinition" id="%ID%" action="get" levels="2" />
```

Note: %ID% should be replaced with the ID of the Query Definition that will be executed with Effectivity Resolution.

8.2.2 Specifying Query Parameter Values for the Query Definition

To execute a Query Definition with Effectivity Resolution, values for the appropriate query parameters must be specified. This can be done using the Query Definition's AML as the starting point.

1. Open the Query Definition AML retrieved in Section 8.2.1.
2. Find the "EffectivityScopeId" qry_QueryParameter item. Add the following XML node:
<value>%value%</value> with the Effectivity Scope Item ID as the value of the node. In the following example, the ID of the "Aras Part BOM Scope" Effectivity Scope is used.

```
1 <Item type="qry_QueryParameter" typeId="1F23B91C84264C6D9DF614DF2833D99D" id="A87B2A2E5C06499EBA565741219C511C">
2   <behavior>fixed</behavior>
3   <config_id keyed_name="A87B2A2E5C06499EBA565741219C511C" type="qry_QueryParameter">A87B2A2E5C06499EBA565741219C511C</config_id>
4   <created_by_id keyed_name="Super User" type="User">AD30A6D8D3B642F5A2AFED1A4B02BEFA</created_by_id>
5   <created_on>2019-04-25T06:16:42</created_on>
6   <generation>1</generation>
7   <id keyed_name="A87B2A2E5C06499EBA565741219C511C" type="qry_QueryParameter">A87B2A2E5C06499EBA565741219C511C</id>
8   <is_current>1</is_current>
9   <is_released>0</is_released>
10  <keyed_name>A87B2A2E5C06499EBA565741219C511C</keyed_name>
11  <major_rev>A</major_rev>
12  <modified_by_id keyed_name="Super User" type="User">AD30A6D8D3B642F5A2AFED1A4B02BEFA</modified_by_id>
13  <modified_on>2019-04-25T06:16:42</modified_on>
14  <name>EffectivityScopeId</name>
15  <new_version>1</new_version>
16  <not_lockable>0</not_lockable>
17  <permission_id keyed_name="Query Definition - In Work" type="Permission" origPermission="08862820B2384F2D9460295">08862820B2384F2D9460295</permission_id>
18  <sort_order>384</sort_order>
19  <source_id keyed_name="PE BomStructure" type="qry_QueryDefinition">F40469675A0C4137B5504B591995D04A</source_id>
20  <value>D1916568FD3C4E5C92FFE42A57B60CC8</value> E.g., id of the 'Aras Part BOM Scope' Effectivity Scope item
21 </Item>
```

Figure 61.

3. Prepare the Effectivity Criteria defined in free-form Boolean language in the following format:

```
<EQ>...</EQ><EQ>...</EQ>...<EQ>...</EQ>
```

For example:

```
<EQ><variable id="ID_OF_MODEL_VARIABLE"/><named-constant
  id="ID_OF_MODEL_ITEM" /></EQ><EQ><variable
  id="ID_OF_UNIT_VARIABLE"/><constant type="int">11</constant></EQ>
```

Note: To understand the Scope structure required and to retrieve the correct IDs for each criterion, it is best to run a request to the `cfg_GetScopeStructure` method. Refer to the Section 5.2.1 of the *Aras Innovator 33 - Configurator Services Programmers Guide* for further details.

4. Escape the XML generated in the previous step to make it XML-safe. This can be done using a text editor or an online XML escape tool. Save the resulting escaped string.

5. Find the "EffectivityCriteriaDefinition" qry_QueryParameter item. Add the following XML node:
<value>%value%</value> with the escaped string defining the prepared Effectivity Criteria as the value of the node, similar to the following example:

```

1 <Item type="qry_QueryParameter" typeId="1F23B91C84264C6D9DF614DF2833D99D" id="968FB53DD6AB404B8073377650458BDD">
2   <behavior>fixed</behavior>
3   <config_id keyed_name="968FB53DD6AB404B8073377650458BDD" type="qry_QueryParameter">968FB53DD6AB404B8073377650458BDD
4   </config_id>
5   <created_by_id keyed_name="Super User" type="User">AD30A6D8D3B642F5A2AFED1A4B02BEFA</created_by_id>
6   <created_on>2019-04-25T06:16:42</created_on>
7   <generation>1</generation>
8   <id keyed_name="968FB53DD6AB404B8073377650458BDD" type="qry_QueryParameter">968FB53DD6AB404B8073377650458BDD</id>
9   <is_current>1</is_current>
10  <is_released>0</is_released>
11  <keyed_name>968FB53DD6AB404B8073377650458BDD</keyed_name>
12  <major_rev>A</major_rev>
13  <modified_by_id keyed_name="Super User" type="User">AD30A6D8D3B642F5A2AFED1A4B02BEFA</modified_by_id>
14  <modified_on>2019-04-25T06:16:42</modified_on>
15  <name>EffectivityCriteriaDefinition</name>
16  <new_version>1</new_version>
17  <not_lockable>0</not_lockable>
18  <permission_id keyed_name="Query Definition - In Work" type="Permission" origPermission=
19  "08862820B2384F2D9460295A5B4CB0F1">08862820B2384F2D9460295A5B4CB0F1</permission_id>
20  <sort_order>512</sort_order>
21  <source_id keyed_name="PE BomStructure" type="qry_QueryDefinition">F40469675A0C4137B5504B591995D04A</source_id>
22  <value>&lt;eq&gt;&lt;variable id=&quot;E0A261A8F65649F0B87DE2B8DB23CC20&quot;/&gt;&lt;constant type=&quot;int
23  &quot;&gt;&lt;11&lt;/constant&gt;&lt;/eq&gt;&lt;/value> E.g., Unit = 11
24 </Item>

```

Figure 62.

6. Find the qry_QueryReference item with no parent_ref_id node inside it. This item represents the entry point, or top node, for traversing a structure. Add the following XML node:
<filter_xml>...</filter_xml> to provide a start condition.

If the Query Definition's top level (root) Item is an ItemType Item (e.g. Part), use:

```

<filter_xml>&lt;condition&gt;
&lt;eq&gt;
&lt;property name=&apos;id&apos; /&gt;
&lt;constant&gt;%ITEM-ID%&lt;/constant&gt;
&lt;/eq&gt;
&lt;/condition&gt;</filter_xml>

```

If the Query Definition's top level (root) Item is a RelationshipType Item (e.g. Part BOM), use:

```

<filter_xml>&lt;condition&gt;
&lt;eq&gt;
&lt;property name=&apos;source_id&apos; /&gt;
&lt;constant&gt;%ITEM-ID%&lt;/constant&gt;
&lt;/eq&gt;
&lt;/condition&gt;</filter_xml>

```

Note: %ITEM-ID% should be replaced with the ID of the Item that is the root item in the structure where effectivity will be applied.

```
<Item type="qry_QueryReference" typeId="F96B52A885B04917A1C34251248E47D6" id="799DC78332E741B3BD9CC114E4E66D8C">
  <behavior>float</behavior>
  <child_ref_id>C367FA04BD74494FA9182CC1BE6031CC</child_ref_id>
  <config_id keyed_name="799DC78332E741B3BD9CC114E4E66D8C" type="qry_QueryReference">799DC78332E741B3BD9CC114E4E66D8C
  </config_id>
  <created_by_id keyed_name="Super User" type="User">AD30A6D8D3B642F5A2AFED1A4B02BEFA</created_by_id>
  <created_on>2020-03-24T05:48:48</created_on>
  <generation>1</generation>
  <id keyed_name="799DC78332E741B3BD9CC114E4E66D8C" type="qry_QueryReference">799DC78332E741B3BD9CC114E4E66D8C</id>
  <is_current>1</is_current>
  <is_released>0</is_released>
  <keyed_name>799DC78332E741B3BD9CC114E4E66D8C</keyed_name>
  <major_rev>A</major_rev>
  <modified_by_id keyed_name="Super User" type="User">AD30A6D8D3B642F5A2AFED1A4B02BEFA</modified_by_id>
  <modified_on>2020-03-24T05:48:48</modified_on>
  <new_version>1</new_version>
  <not_lockable>0</not_lockable>
  <permission_id keyed_name="Query Definition - In Work" type="Permission" origPermission=
  "08862820B2384F2D9460295A5B4CB0F1">08862820B2384F2D9460295A5B4CB0F1</permission_id>
  <sort_order>128</sort_order>
  <source_id keyed_name="PE_BomStructure" type="qry_QueryDefinition">F40469675A0C4137B5504B591995D04A</source_id>
  <filter_xml>&lt;condition&gt;
    &lt;eq&gt;
      &lt;property name='source_id' /&gt;
      &lt;constant&gt;5957740D88BA420FA40CE2B98A610F9E&lt;/constant&gt;
    &lt;/eq&gt;
  &lt;/condition&gt;&lt;/filter_xml>
</Item>
```

E.g., [Part BOM].source_id = '5957740D88BA420FA40CE2B98A610F9E'

Figure 63.

7. Save the AML response.

8.2.3 Query Definition Execution

1. Open the saved AML response from step 7 of Section 8.2.2.
2. Find the root qry_QueryDefinition item and set the value of its action attribute to "qry_ExecuteQueryDefinition".

```
1 <Item type="qry_QueryDefinition" typeId="0421B558114B4B22886C0F4206C923EE" id="F40469675A0C4137B5504B591995D04A"
  action="qry_ExecuteQueryDefinition">
2   <config_id keyed_name="PE_BomStructure" type="qry_QueryDefinition">F40469675A0C4137B5504B591995D04A
3   </config_id>
4   <created_by_id keyed_name="Super User" type="User">AD30A6D8D3B642F5A2AFED1A4B02BEFA</created_by_id>
5   <created_on>2019-04-25T06:16:41</created_on>
6   <current_state name="Start" keyed_name="Start" type="Life Cycle State">A19F8DFC829349B79928EE674977ED5F
7   </current_state>
8   <generation>1</generation>
9   <id keyed_name="PE_BomStructure" type="qry_QueryDefinition">F40469675A0C4137B5504B591995D04A</id>
10  <is_current>1</is_current>
11  <is_released>0</is_released>
12  <keyed_name>PE_BomStructure</keyed_name>
13  <major_rev>A</major_rev>
14  <modified_by_id keyed_name="Super User" type="User">AD30A6D8D3B642F5A2AFED1A4B02BEFA</modified_by_id>
15  <modified_on>2019-04-25T06:16:42</modified_on>
16  <new_version>1</new_version>
17  <not_lockable>0</not_lockable>
18  <permission_id keyed_name="Query Definition - In Work" type="Permission" origPermission=
  "08862820B2384F2D9460295A5B4CB0F1">08862820B2384F2D9460295A5B4CB0F1</permission_id>
19  <state>Start</state>
20  <name>PE_BomStructure</name>
```

Figure 64.

3. The resulting AML can now be run to execute the Query Definition with Effectivity Resolution applied using the set parameters for the resolution.